

Введение	3
1 Теоретическая часть	5
1.1 Описание предметной области	5
1.2 Анализ существующей ситуации	6
1.3 Постановка задачи	9
1.4 Анализ существующих разработок и обоснование актуальности нового проекта	11
2 Специальная часть	13
2.1 Выбор технологий и инструментальных средств	13
2.1.1 Выбор подхода к работе	13
2.1.2 Выбор языка программирования и среды разработки	14
2.2 Разработка спецификаций	17
2.2.1 Разработка диаграмм вариантов использования	17
2.2.2 Разработка диаграмм последовательностей системы	17
2.2.3 Разработка диаграмм пакетов	19
2.2.4 Разработка диаграмм классов	20
2.2.5 Разработка диаграммы состояний	20
2.2.6 Разработка диаграммы деятельности	22
2.2.7 Разработка инфологической модели базы данных	23
2.2.8 Разработка физической модели базы данных	23
2.2.9 Разработка диаграммы размещения	24
2.3 Проектирование программного модуля	25

					ДП(Р).09.02.07 ПЗ				
Изм.	Лист	№ докум.	Подпись	Дата					
Разраб.		Газзаев А.Г.			Разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом Пояснительная записка	Лит.	Лист	Листов	
Провер.		Тагизаде С.Б.							
							1	68	
Н.контр		Каштанова О.Г.				БПОУ ВО «Череповецкий химико- технологический колледж» Группа 81/2022			
Утв.		Мараков В.В.							

2.3.1 Разработка алгоритмов реализации основных функций программного обеспечения	25
2.3.2 Проектирование пользовательского интерфейса	27
2.4 Реализация программного обеспечения на выбранном языке программирования и в выбранной среде разработки	29
2.5 Выбор стратегии тестирования, разработка тестов, тестирование и отладка программного обеспечения	34
3 Экономическая часть	38
Охрана труда	44
Заключение	55
Список используемых источников	57
Приложение А. Графическая часть	59
Приложение Б. Интерфейс программного обеспечения	63
Приложение В. Фрагмент листинга программы	65

Введение

Современный этап развития информационных технологий характеризуется активным формированием цифровых экосистем, объединяющих множество сервисов в рамках единой технологической платформы. Крупные мировые компании, такие как Google, Яндекс и Microsoft, строят свои продукты на основе распределённых архитектур, обеспечивающих масштабируемость, отказоустойчивость и централизованное управление доступом пользователей. Ключевым элементом подобных решений является ядро цифровой экосистемы, реализованное на основе микросервисной архитектуры с единой системой аутентификации и API-шлюзом [2] [5].

В условиях роста количества онлайн-сервисов и увеличения числа пользователей особую значимость приобретает обеспечение безопасного и унифицированного взаимодействия между компонентами системы. Отсутствие централизованной аутентификации и единой точки входа приводит к дублированию функциональности, усложнению сопровождения, снижению уровня безопасности и ограничению возможностей масштабирования. Разрозненные сервисы без общей архитектурной основы затрудняют интеграцию новых компонентов и увеличивают нагрузку на разработчиков и администраторов инфраструктуры.

Разработка ядра цифровой экосистемы на основе микросервисной архитектуры позволяет создать гибкую и расширяемую платформу, обеспечивающую централизованное управление пользователями, маршрутизацию запросов через API-шлюз, контроль доступа и безопасное межсервисное взаимодействие. Такое решение формирует основу для построения различных цифровых сервисов, которые могут развиваться независимо друг от друга, сохраняя при этом целостность общей системы.

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						3
Изм.	Лист	№ докум.	Подпись	Дата		

В настоящее время тема проектирования микросервисных платформ с единой аутентификацией и API-шлюзом является одной из приоритетных в области разработки корпоративных и облачных решений. Использование современных архитектурных подходов позволяет повысить устойчивость информационных систем, упростить их масштабирование и обеспечить высокий уровень информационной безопасности.

Целью дипломной работы является разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом. Для достижения поставленной цели необходимо решить следующие задачи:

- изучить современные подходы к построению микросервисных архитектур и цифровых платформ;
- провести анализ предметной области и существующих технических решений;
- определить функциональные и нефункциональные требования к системе;
- разработать архитектуру ядра цифровой экосистемы;
- спроектировать программные компоненты системы (службу аутентификации, API-шлюз, механизмы взаимодействия сервисов);
- реализовать программное решение;
- провести тестирование и оценку эффективности разработанной системы.

Внедрение разработанного решения позволит создать универсальную основу для построения цифровых сервисов различного назначения, повысить управляемость и безопасность инфраструктуры, а также обеспечить возможность дальнейшего масштабирования и развития экосистемы в условиях цифровой трансформации.

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

1 Теоретическая часть

1.1 Описание предметной области

Предметной областью данного программного обеспечения является ядро цифровой экосистемы, построенной на основе микросервисной архитектуры, а также организация централизованной аутентификации, авторизации и взаимодействия сервисов через API-шлюз.

Цифровая экосистема представляет собой совокупность взаимосвязанных сервисов, объединённых общей архитектурной платформой и едиными механизмами управления доступом. В современных условиях такие экосистемы используются для предоставления широкого спектра цифровых услуг в рамках единой инфраструктуры. Ключевым элементом экосистемы является её ядро, обеспечивающее централизованную обработку запросов, проверку прав доступа пользователей, маршрутизацию трафика и координацию взаимодействия между микросервисами.

Организация работы цифровой экосистемы включает в себя решение задач аутентификации и авторизации пользователей, управления ролями и правами доступа, безопасного обмена данными между сервисами, мониторинга состояния компонентов системы и обеспечения масштабируемости. Важной частью является формирование единой точки входа в систему посредством API-шлюза, что позволяет упростить клиентское взаимодействие и повысить уровень информационной безопасности [20].

Одной из актуальных проблем в области разработки распределённых систем является сложность обеспечения согласованной работы множества сервисов при сохранении гибкости архитектуры. При отсутствии централизованного ядра возникают трудности с управлением доступом, дублирование функциональности, усложнение сопровождения и масштабирования системы. Кроме того, разрозненные механизмы аутентификации и прямое взаимодействие сервисов повышают риски безопасности и увеличивают затраты на поддержку инфраструктуры.

					ДП(Р).09.02.07 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

Разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом позволяет решить указанные проблемы. Такое решение обеспечивает модульность, расширяемость и отказоустойчивость системы, централизованное управление пользователями и безопасную маршрутизацию запросов. Оно также создаёт основу для дальнейшего подключения новых сервисов без существенной переработки существующей архитектуры.

Данное программное обеспечение может быть востребовано в организациях различного масштаба и сферы деятельности – от стартапов до крупных корпоративных структур, развивающих собственные цифровые платформы. Разработанное ядро может быть интегрировано в существующую IT-инфраструктуру, обеспечивая технологическую основу для построения и развития современной цифровой экосистемы.

1.2 Анализ существующей ситуации

Существующая ситуация в области разработки и эксплуатации цифровых сервисов характеризуется высокой степенью фрагментации архитектурных решений. В разных организациях и проектах используются различные подходы к построению распределённых систем, однако можно выделить ряд общих проблем, которые могут быть эффективно решены за счёт разработки ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом:

1. Разрозненность сервисов и отсутствие единой точки входа. Во многих системах сервисы разрабатываются и развёртываются независимо друг от друга без централизованного механизма маршрутизации запросов. Это приводит к усложнению клиентского взаимодействия, необходимости прямого обращения к различным сервисам и повышению рисков безопасности.
2. Отсутствие централизованной аутентификации и управления доступом. Использование отдельных механизмов входа в каждом сервисе

					ДП(Р).09.02.07 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

приводит к дублированию функциональности, усложнению сопровождения и повышенной вероятности ошибок в реализации безопасности. Отсутствие единой системы управления ролями и правами доступа затрудняет контроль над пользовательскими действиями.

3. Сложности масштабирования и сопровождения системы. При росте количества сервисов и пользователей нагрузка на инфраструктуру увеличивается. Без продуманной микросервисной архитектуры и API-шлюза возникают трудности с балансировкой нагрузки, обновлением компонентов и обеспечением отказоустойчивости.
4. Недостаточная прозрачность и управляемость инфраструктуры. Отсутствие централизованного логирования, мониторинга и унифицированной обработки ошибок усложняет выявление проблем, анализ инцидентов и контроль состояния системы в целом.
5. Трудности интеграции новых сервисов. Без наличия архитектурного ядра подключение новых модулей требует значительных изменений в существующих компонентах, что снижает гибкость системы и увеличивает затраты на развитие платформы.

Для обоснования необходимости разработки ядра цифровой экосистемы были построены две сравнительные диаграммы: модель существующего процесса AS IS и модель целевого процесса TO BE.

Диаграмма AS IS отражает работу разрозненных сервисов до внедрения разрабатываемого программного обеспечения. В данной модели каждый сервис имеет собственный механизм аутентификации, отдельную обработку запросов и индивидуальные правила доступа. Такой подход приводит к дублированию функциональности, усложнению сопровождения, повышению нагрузки на разработчиков и снижению уровня безопасности. Диаграмма AS IS представлена на рисунке 1.

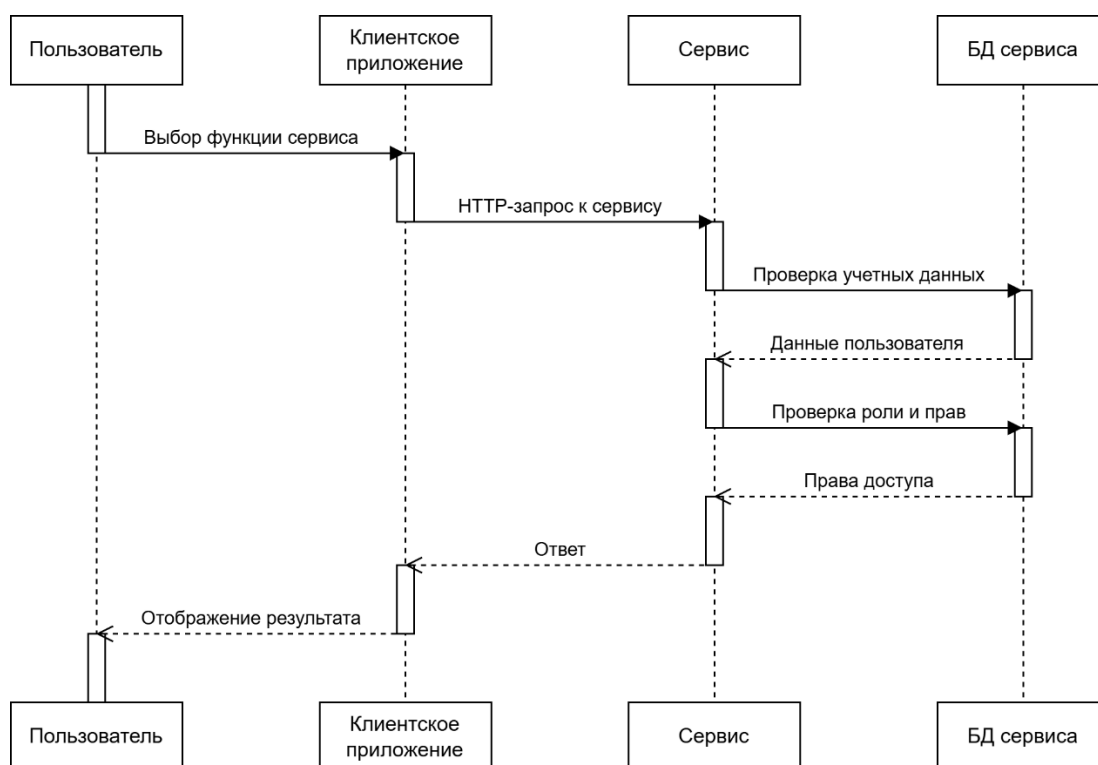


Рисунок 1 – Диаграмма AS IS

Диаграмма TO BE показывает процесс обработки запросов после внедрения ядра цифровой экосистемы. В новой модели все клиентские запросы проходят через единый API-шлюз, а аутентификация и авторизация выполняются централизованно. Это позволяет упростить архитектуру, повысить безопасность, обеспечить единый механизм управления доступом и облегчить подключение новых микросервисов. Диаграмма TO BE представлена на рисунке 2.

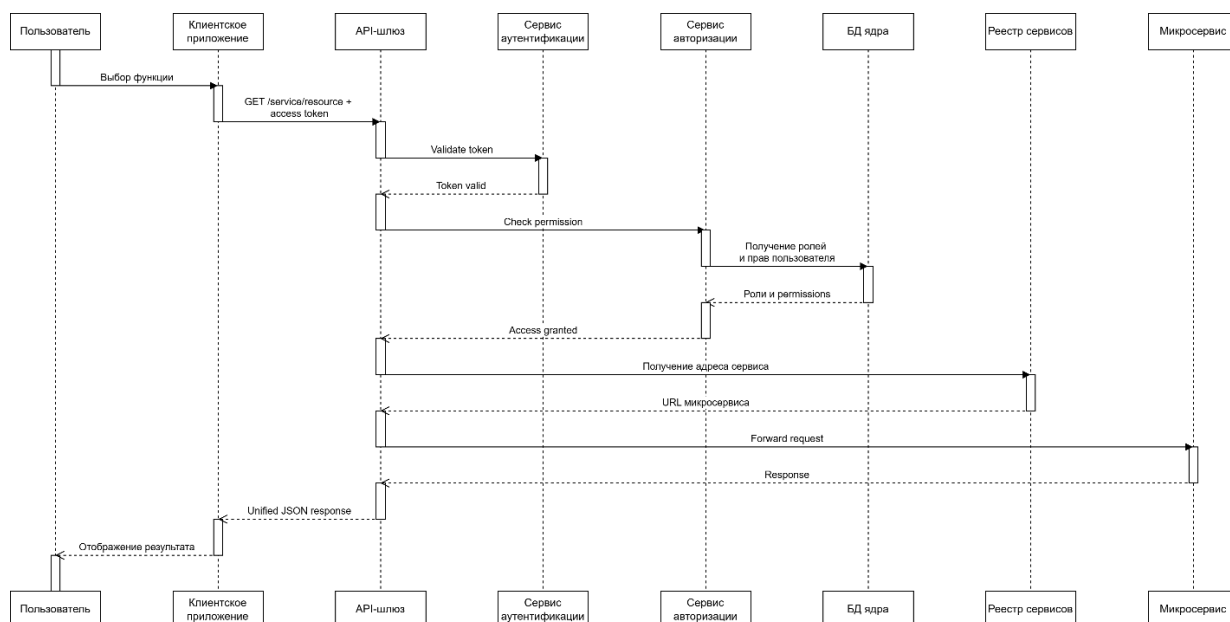


Рисунок 2 – Диаграмма TO BE

Сравнение диаграмм показывает, что внедрение ядра цифровой экосистемы позволяет перейти от разрозненной и трудно сопровождаемой архитектуры к централизованной, управляемой и масштабируемой модели. Использование API-шлюза, единой аутентификации и централизованной авторизации обеспечивает более высокий уровень безопасности, снижает дублирование функциональности и упрощает дальнейшее развитие системы.

1.3 Постановка задачи

Целью дипломной работы является разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом. Одним из ключевых требований к разрабатываемой системе является обеспечение унифицированного взаимодействия между микросервисами и внешними клиентскими приложениями с использованием стандартизированных форматов обмена данными, таких как JSON, что позволяет упростить интеграцию и повысить совместимость с другими системами и платформами.

Для достижения поставленной цели необходимо последовательно решить ряд задач, каждая из которых реализуется с применением соответствующих методов и подходов.

На первом этапе выполняется изучение современных подходов к построению микросервисных архитектур и цифровых платформ. Для этого проводится анализ научной литературы, технической документации и практик разработки, включая принципы построения распределённых систем, контейнеризации, сервисной декомпозиции и организации взаимодействия между сервисами.

Далее осуществляется анализ предметной области и существующих технических решений. В рамках данного этапа рассматриваются современные платформы и инструменты, реализующие API-шлюзы, системы аутентификации и управления доступом. Анализ позволяет выявить их преимущества, недостатки и особенности реализации, которые будут учтены при проектировании собственной системы.

На следующем этапе производится определение функциональных и нефункциональных требований к разрабатываемой системе. Функциональные требования описывают основные возможности системы, такие как регистрация и аутентификация пользователей, маршрутизация запросов и взаимодействие микросервисов. Нефункциональные требования включают требования к производительности, масштабируемости, безопасности, отказоустойчивости и удобству сопровождения [7].

После формализации требований разрабатывается архитектура ядра цифровой экосистемы. На данном этапе определяется структура системы, выделяются основные компоненты, такие как API-шлюз, служба аутентификации, сервис авторизации и микросервисы, а также описываются их взаимодействия. При проектировании используются принципы микросервисной архитектуры, включая слабую связанность компонентов и независимость их развертывания [6].

Следующим шагом является проектирование программных компонентов системы. Для этого разрабатываются модели данных, интерфейсы взаимодействия сервисов, а также логика работы ключевых модулей, включая механизм аутентификации пользователей, генерацию и проверку токенов, обработку и маршрутизацию API-запросов. Также определяется формат обмена данными (JSON) и структура API [18].

На этапе реализации производится разработка программного решения с использованием выбранных технологий. Реализуются основные компоненты системы, включая API-шлюз, сервис аутентификации и базовые микросервисы. Особое внимание уделяется обеспечению корректного взаимодействия между компонентами, обработке ошибок и логированию.

Заключительным этапом является тестирование и оценка эффективности разработанной системы. Проводится проверка корректности работы всех компонентов, тестирование сценариев аутентификации и авторизации, а также оценка производительности и устойчивости системы при обработке запросов. По результатам тестирования формулируются выводы о качестве разработанного решения и его соответствии поставленным требованиям.

1.4 Анализ существующих разработок и обоснование актуальности нового проекта

На рынке современных цифровых платформ уже существуют решения, ориентированные на построение распределённых систем и микросервисных экосистем с централизованной аутентификацией и API-шлюзом. Эти решения обеспечивают унифицированное взаимодействие между сервисами, управление доступом пользователей и маршрутизацию запросов, но различаются по масштабу, архитектуре и набору функций.

Например, компания Amazon Web Services (AWS) предлагает платформу AWS App Mesh и сервисы API Gateway, которые позволяют организовать взаимодействие микросервисов, централизованно управлять доступом и обеспечивать отказоустойчивость распределённых приложений.

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						11
Изм.	Лист	№ докум.	Подпись	Дата		

Эти решения поддерживают маршрутизацию запросов, мониторинг состояния сервисов и масштабирование инфраструктуры [19].

Другой пример – платформа Kong Inc., предоставляющая API-шлюз с функциями управления доступом, аутентификации, лимитирования запросов и логирования. Решение ориентировано на обеспечение безопасного и управляемого взаимодействия между клиентскими приложениями и микросервисами, а также на упрощение интеграции новых сервисов в существующую архитектуру.

Существуют также комплексные решения для построения корпоративных цифровых экосистем, такие как Red Hat OpenShift, которые предоставляют инфраструктуру для контейнеризации и оркестрации микросервисов, управление ролями и политиками безопасности, а также инструменты мониторинга и масштабирования приложений [16].

Анализ существующих решений показывает, что современные платформы обеспечивают гибкость архитектуры, безопасность, масштабируемость и централизованное управление доступом. Однако большинство из них ориентированы на крупные корпоративные инфраструктуры или облачные решения, что делает их сложными для внедрения в качестве универсального ядра цифровой экосистемы для разнопрофильных сервисов.

Таким образом, разработка собственного ядра цифровой экосистемы с микросервисной архитектурой, единой аутентификацией и API-шлюзом позволит создать универсальное, безопасное и масштабируемое решение, обеспечивающее простоту интеграции новых сервисов, централизованное управление пользователями и эффективное взаимодействие между компонентами платформы.

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						12
Изм.	Лист	№ докум.	Подпись	Дата		

2 Специальная часть

2.1 Выбор технологий и инструментальных средств

2.1.1 Выбор подхода к разработке

Для разработки ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом наиболее подходящим является компонентно-ориентированный и модульный подход с элементами объектно-ориентированного проектирования.

При создании такой системы важно обеспечить чёткое разделение ответственности между отдельными микросервисами и компонентами ядра, такими как служба аутентификации, API-шлюз, сервис маршрутизации, мониторинга и логирования. Каждый компонент должен иметь свои чётко определённые функции и интерфейсы взаимодействия, что позволяет легко масштабировать систему, подключать новые сервисы и обеспечивать отказоустойчивость [10].

В рамках компонентно-ориентированного подхода каждый микросервис реализуется как независимый модуль с собственной логикой, данными и API. При этом можно использовать объектно-ориентированные принципы для внутренней организации кода: классы и объекты помогают структурировать функциональность, управлять состоянием сервиса и реализовывать безопасное взаимодействие с другими модулями. Такой подход повышает масштабируемость, гибкость и удобство сопровождения системы, а также упрощает тестирование и отладку отдельных компонентов [4].

Напротив, использование чисто структурного подхода в разработке распределённых систем с микросервисами может привести к сложностям: повторение кода, тесная связанность компонентов и затруднённое масштабирование, что критично для больших экосистем с высокой нагрузкой.

Таким образом, комбинированный подход – компонентно-ориентированная архитектура с применением принципов ООП внутри микросервисов, является оптимальным для разработки ядра цифровой

					ДП(Р).09.02.07 ПЗ	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		

экосистемы. Он обеспечивает необходимую гибкость, отказоустойчивость, масштабируемость и упрощает интеграцию новых сервисов и технологий в платформу.

2.1.2 Выбор языка программирования и среды разработки

Для разработки ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом необходимо обоснованно выбрать языки программирования, фреймворки и инструменты разработки. Выбор технологий должен основываться на сравнительном анализе существующих решений с учётом требований к масштабируемости, производительности, безопасности и удобству сопровождения.

На этапе анализа были рассмотрены несколько популярных языков программирования для серверной разработки и построения микросервисных систем, включая Python, Java (Spring Boot), Node.js и C# (.NET).

Язык Java и платформа Spring Boot широко применяются при разработке корпоративных систем благодаря высокой производительности, развитой экосистеме и встроенным механизмам безопасности. Однако они требуют большего объёма кода и обладают более высоким порогом вхождения, что может усложнить и замедлить разработку.

Node.js обеспечивает высокую производительность при обработке большого количества асинхронных запросов и активно используется при разработке API. Тем не менее, динамическая типизация JavaScript может приводить к ошибкам на этапе выполнения и усложнять поддержку крупных проектов.

Платформа .NET (C#) предоставляет мощные инструменты разработки, высокую производительность и хорошую интеграцию с корпоративными решениями. Однако она в большей степени ориентирована на экосистему Microsoft и может требовать дополнительных ресурсов для развертывания.

Язык Python, в свою очередь, отличается простотой синтаксиса, высокой скоростью разработки и наличием большого количества библиотек для работы

					ДП(Р).09.02.07 ПЗ	Лист
						14
Изм.	Лист	№ докум.	Подпись	Дата		

с веб-сервисами, безопасностью и взаимодействием компонентов распределённых систем. Несмотря на несколько меньшую производительность по сравнению с Java или C#, Python является оптимальным выбором для разработки прототипов и систем средней нагрузки, особенно при использовании современных асинхронных фреймворков [14].

В результате проведённого анализа в качестве основного языка программирования выбран Python [1].

Для разработки REST API также были рассмотрены фреймворки FastAPI, Flask и Django. Фреймворк Django является мощным решением с большим количеством встроенных возможностей, однако он избыточен для микросервисной архитектуры и менее гибок в контексте лёгких сервисов.

Flask представляет собой минималистичный фреймворк, позволяющий гибко настраивать архитектуру приложения, однако требует подключения большого количества сторонних компонентов.

FastAPI сочетает простоту разработки, высокую производительность за счёт асинхронной обработки запросов и встроенную поддержку валидации данных и генерации документации. В связи с этим он является наиболее предпочтительным для реализации микросервисов [12].

Для обеспечения изоляции компонентов системы и упрощения процесса развертывания были рассмотрены подходы виртуализации и контейнеризации. Использование Docker позволяет упаковывать каждый микросервис в отдельный контейнер, обеспечивая независимость компонентов и удобство масштабирования. Для управления несколькими контейнерами целесообразно использовать Docker Compose, а при необходимости масштабирования и управления кластером – Kubernetes [8].

В качестве системы управления базами данных были рассмотрены PostgreSQL и MySQL. Обе СУБД являются надёжными и широко используемыми решениями. PostgreSQL обладает более широкими возможностями расширения, поддержкой сложных типов данных и лучшей

работой с транзакциями, что делает её предпочтительной для данной системы [13].

Для повышения производительности и уменьшения задержек при обработке запросов предлагается использовать Redis в качестве системы кэширования, в частности для хранения токенов и часто запрашиваемых данных [15].

В качестве среды разработки выбрана Visual Studio Code, которая обеспечивает поддержку Python, интеграцию с Docker, системой контроля версий Git, а также предоставляет удобные инструменты для отладки, тестирования и сопровождения кода.

Основные преимущества выбранных технологий заключаются в следующем:

1. Гибкость и масштабируемость. Использование микросервисной архитектуры и контейнеризации позволяет легко расширять систему и добавлять новые компоненты без изменения существующих.
2. Надёжность и отказоустойчивость. Изоляция сервисов и возможность их независимого развертывания повышают устойчивость системы к сбоям.
3. Высокая скорость разработки. Python и FastAPI позволяют быстро реализовывать функциональность при сохранении читаемости и структурированности кода.
4. Удобство интеграции. Использование REST API и формата JSON обеспечивает простое взаимодействие между сервисами и внешними приложениями [17].
5. Развитая экосистема. Выбранные технологии имеют широкую поддержку сообщества и большое количество готовых решений.

2.2 Разработка спецификаций

2.2.1 Разработка диаграмм вариантов использования

Диаграмма вариантов использования отражает функции ядра платформы: аутентификацию, авторизацию, маршрутизацию запросов, управление сервисами и администрирование. Она представлена на рисунке 3 [3].

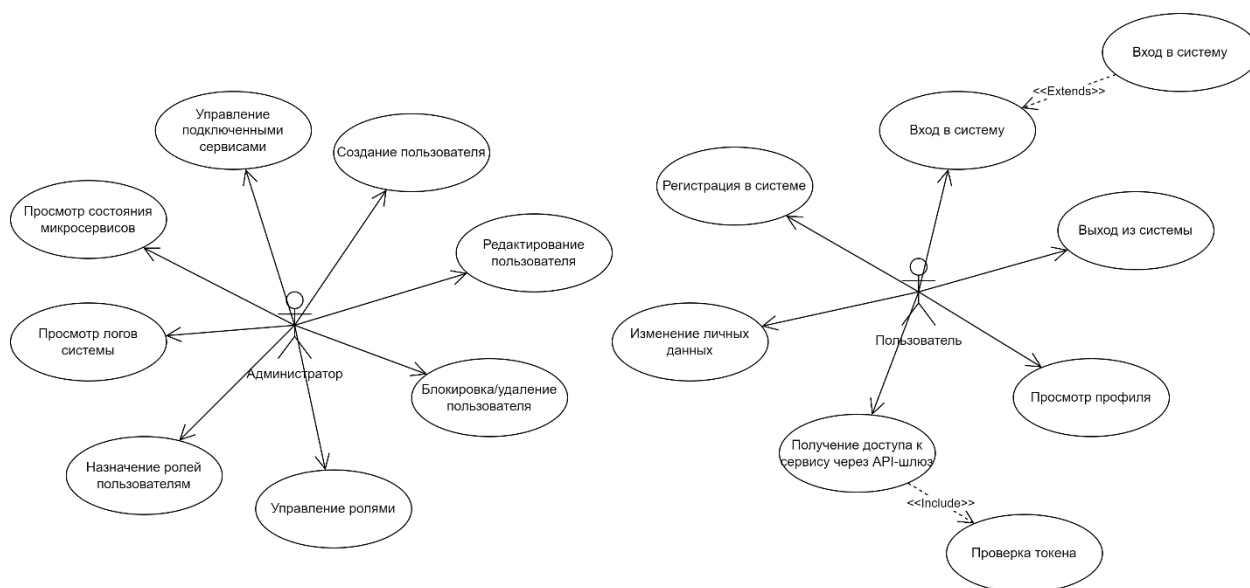


Рисунок 3 – Диаграмма вариантов использования

2.2.2 Разработка диаграмм последовательностей системы

Диаграмма последовательности показывает какой путь проходит запрос от пользователя через все ключевые компоненты ядра. На рисунке 4 представлена диаграмма последовательности аутентификации пользователя.

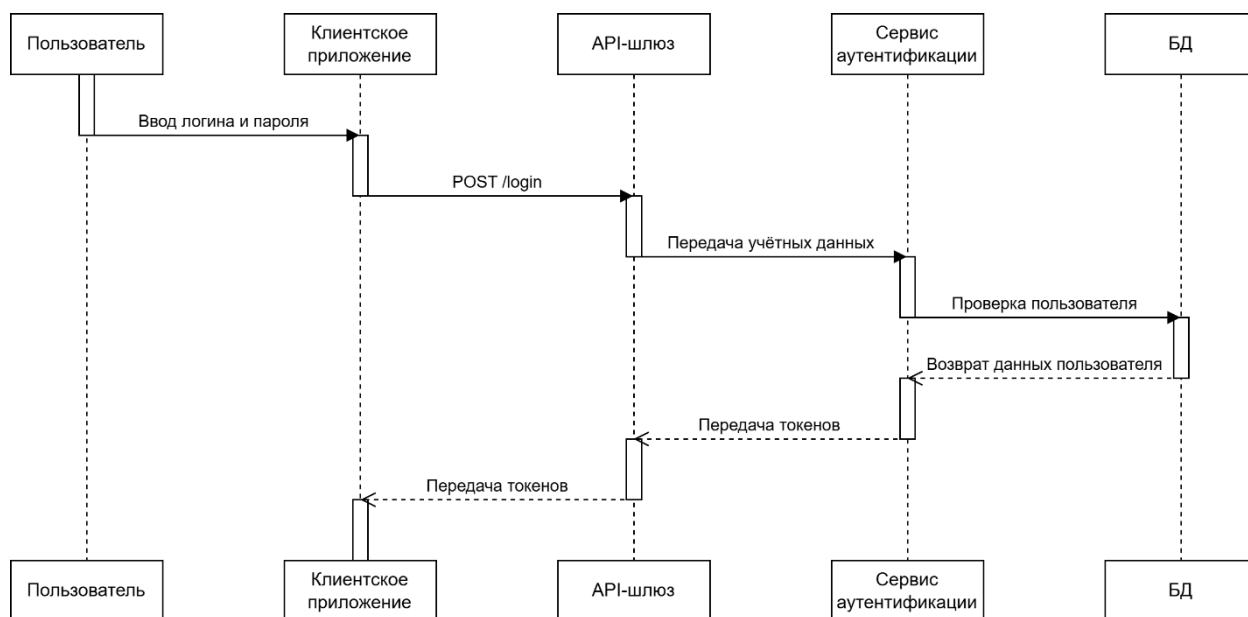


Рисунок 4 – Диаграмма последовательности аутентификации

На рисунке 5 представлена диаграмма последовательности доступа к микросервису.

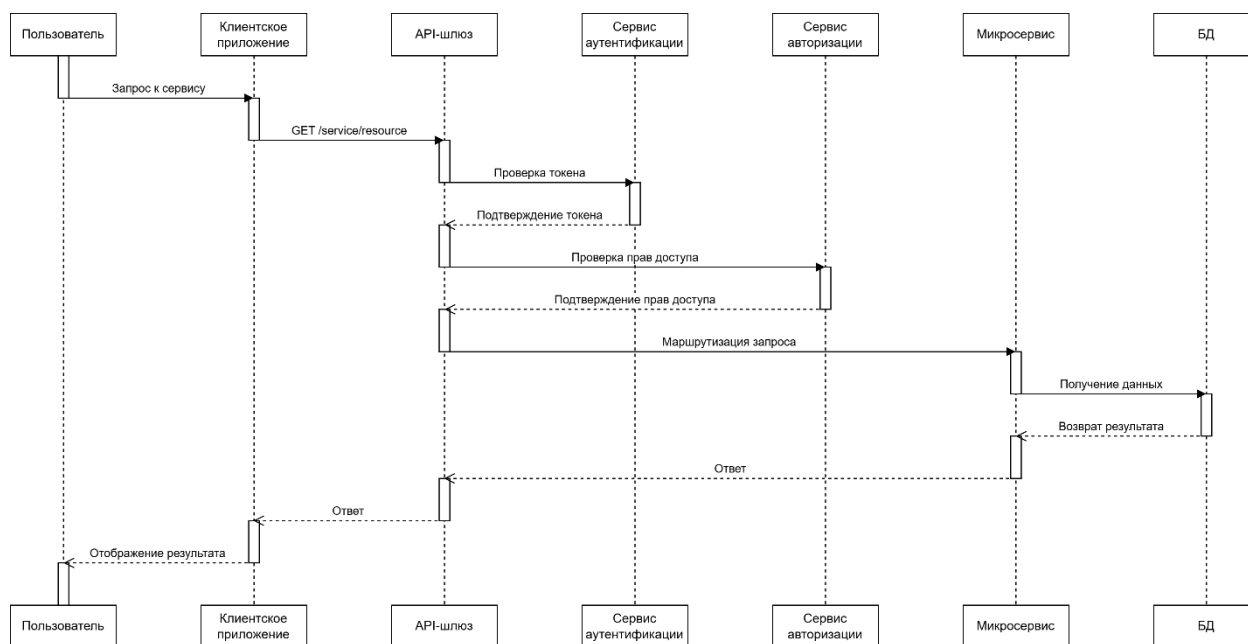


Рисунок 5 – Диаграмма последовательности доступа к микросервису

2.2.3 Разработка диаграммы пакетов

Диаграмма пакетов показывает архитектурную декомпозицию, а также логическую структуру системы и зависимости между её крупными модулями. Диаграмма пакетов представлена на рисунке 6.

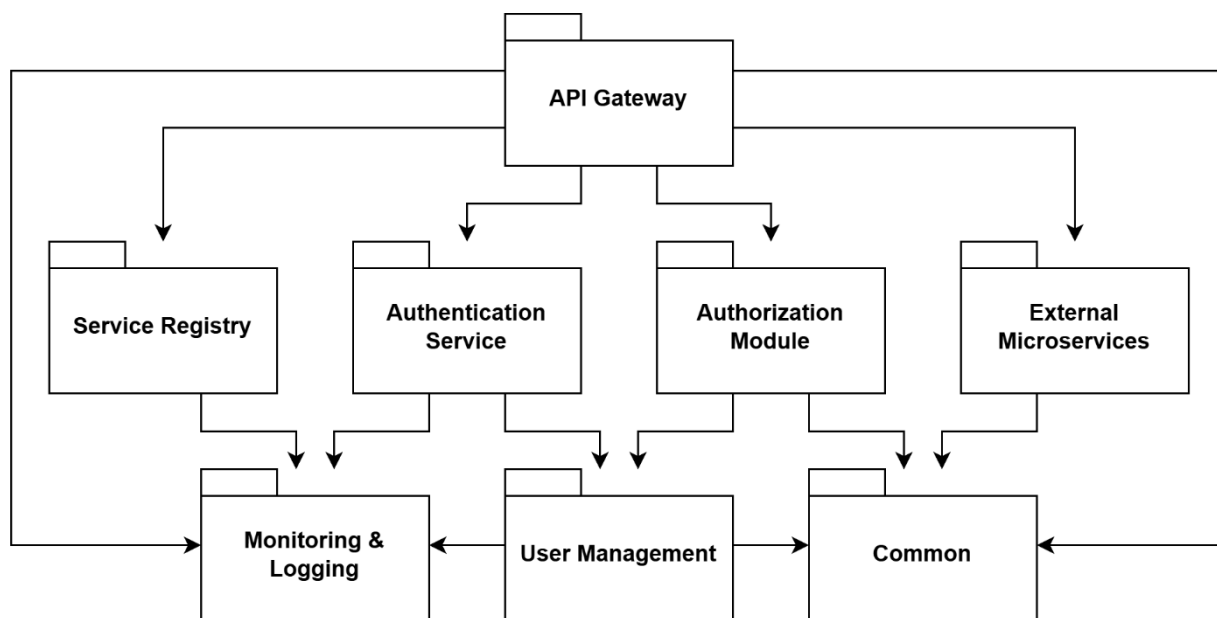


Рисунок 6 – Диаграмма пакетов

Пакеты на диаграмме:

1. «API Gateway» – приём всех входящих запросов, проверка токенов, маршрутизация запросов, ограничение частоты запросов, централизованная обработка ошибок.
2. «Authentication Service» – регистрация пользователей, аутентификация, генерация Access/Refresh токенов, обновление токенов, проверка валидности токенов.
3. «Authorization Module» – управление ролями, назначение прав, проверка доступа к ресурсам, реализация RBAC.
4. «User Management» – CRUD пользователей, хранение профилей, блокировка пользователей, управление статусами.
5. «Service Registry» – регистрация микросервисов, хранение информации о сервисах, предоставление данных для маршрутизации.

Изм.	Лист	№ докум.	Подпись	Дата

ДП(Р).09.02.07 ПЗ

Лист

19

6. «Monitoring & Logging» – логирование запросов, сбор метрик, Health-check сервисов, отслеживание ошибок.
7. «Common» – общие модели данных, DTO, утилиты, константы, обработчики исключений.
8. «External Microservices» – бизнес-сервисы, подключённые к ядру, не входят в ядро напрямую, взаимодействуют только через API Gateway.

2.2.4 Разработка диаграммы классов

Диаграмма классов отражает внутреннюю структуру ключевых компонентов ядра. Она представлена на рисунке 7.

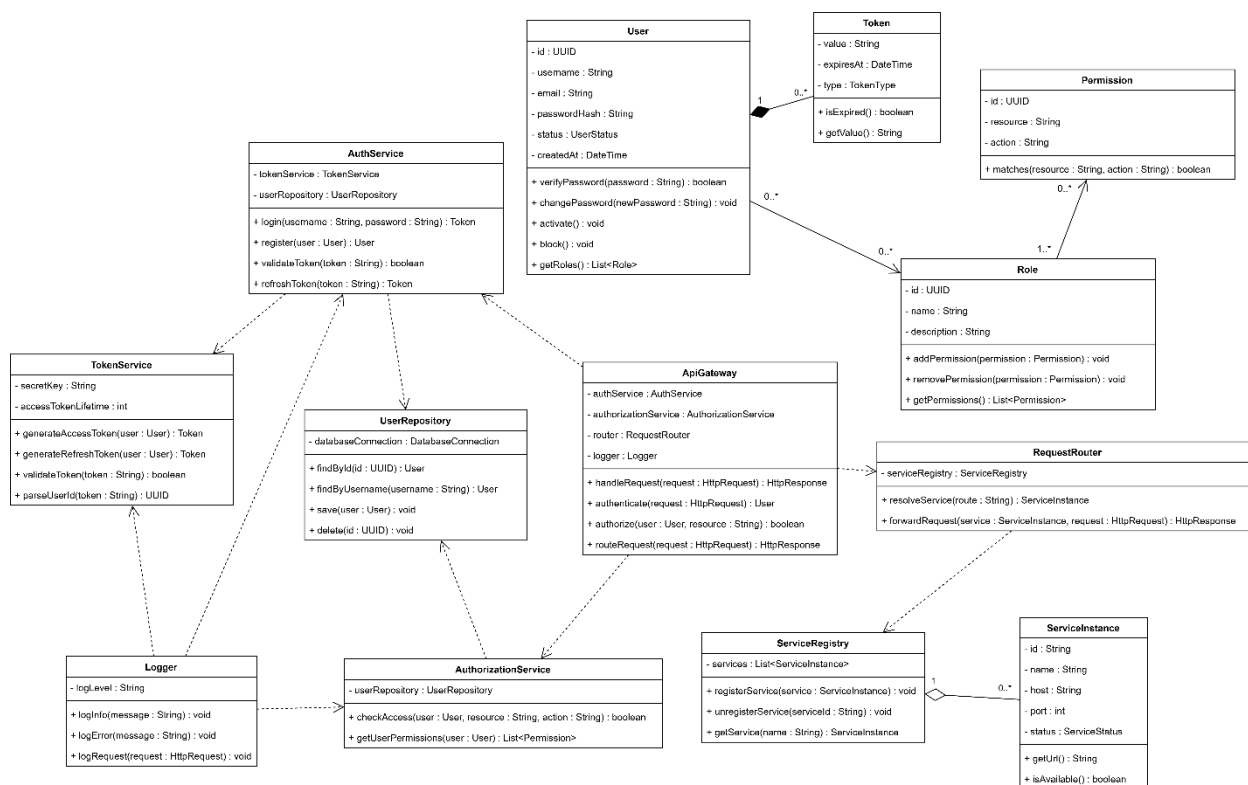


Рисунок 7 – Диаграмма классов

2.2.5 Разработка диаграммы состояний

Диаграмма состояний описывает изменение состояния запроса при его обработке системой. Она показывает, какие этапы проходит запрос в ядре цифровой экосистемы и какие условия вызывают переход между состояниями. Диаграмма состояний представлена на рисунке 8.

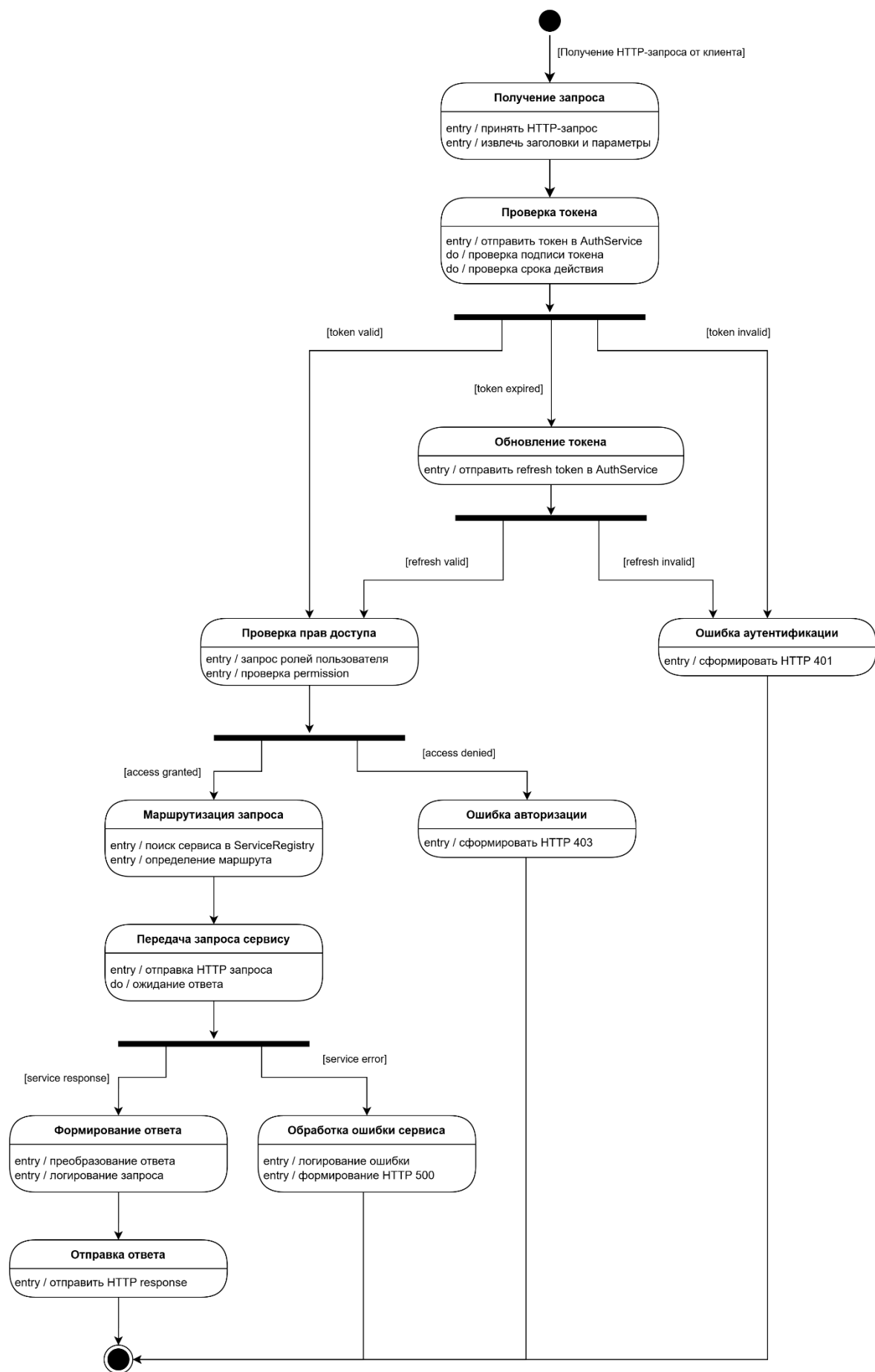


Рисунок 8 – Диаграмма состояний

Изм.	Лист	№ докум.	Подпись	Дата

ДП(Р).09.02.07 ПЗ

Лист

21

2.2.6 Разработка диаграммы деятельности

Диаграмма деятельности описывает процесс обработки пользовательского запроса в системе. Она показывает последовательность действий, принимаемые решения, возможные ветвления и взаимодействие компонентов системы. Диаграмма деятельности представлена на рисунке 9.

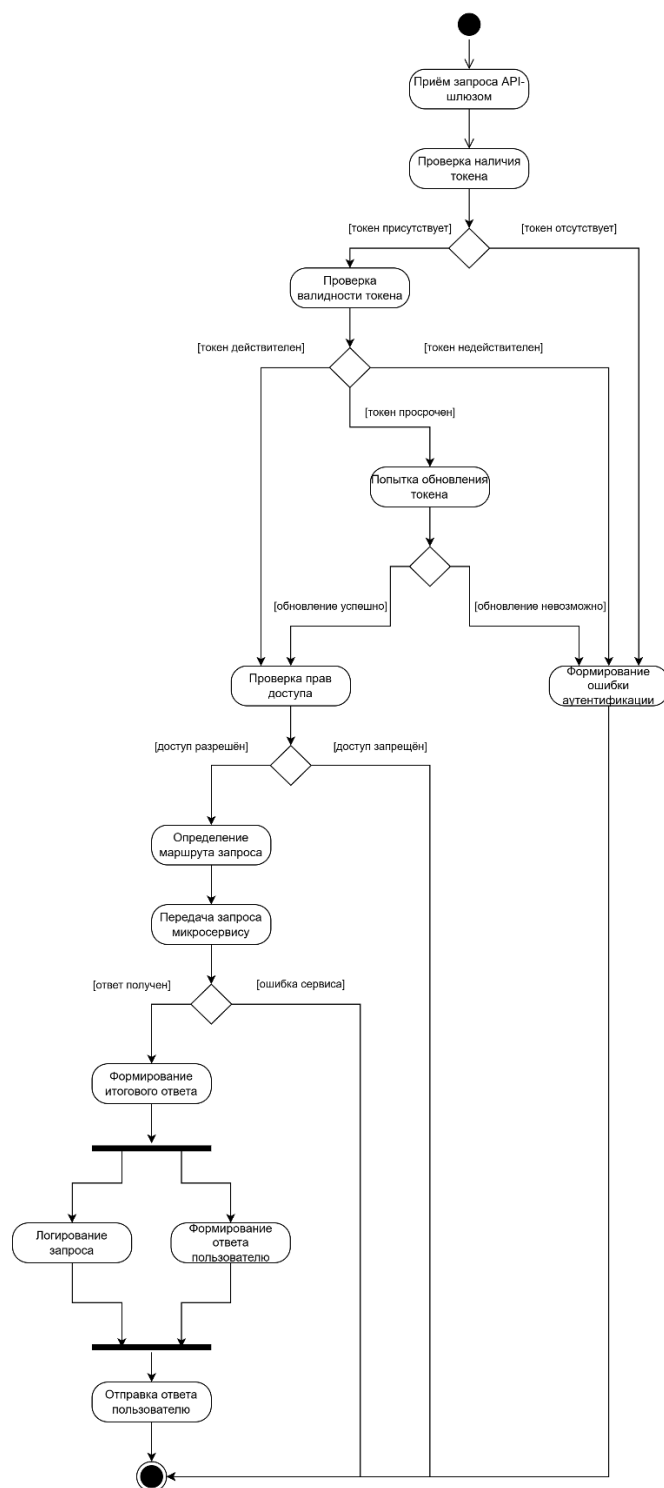


Рисунок 9 – Диаграмма деятельности

2.2.7 Разработка инфологической модели базы данных

Инфологическая модель – это абстрактное представление о данных, которые будут храниться в базе данных. Она описывает данные и их связи между собой, но не учитывает способ их хранения. Эта модель используется для определения структуры базы данных и ее функциональности.

Инфологическая модель базы данных представлена на рисунке 10.



Рисунок 10 – Инфологическая модель базы данных

2.2.8 Разработка физической модели базы данных

Физическая модель базы данных предназначена для хранения данных о пользователях системы, их ролях и правах доступа, токенах аутентификации, а также информации о зарегистрированных микросервисах. База данных обеспечивает централизованное управление безопасностью и инфраструктурой системы. Физическая модель базы данных представлена на рисунке 11.

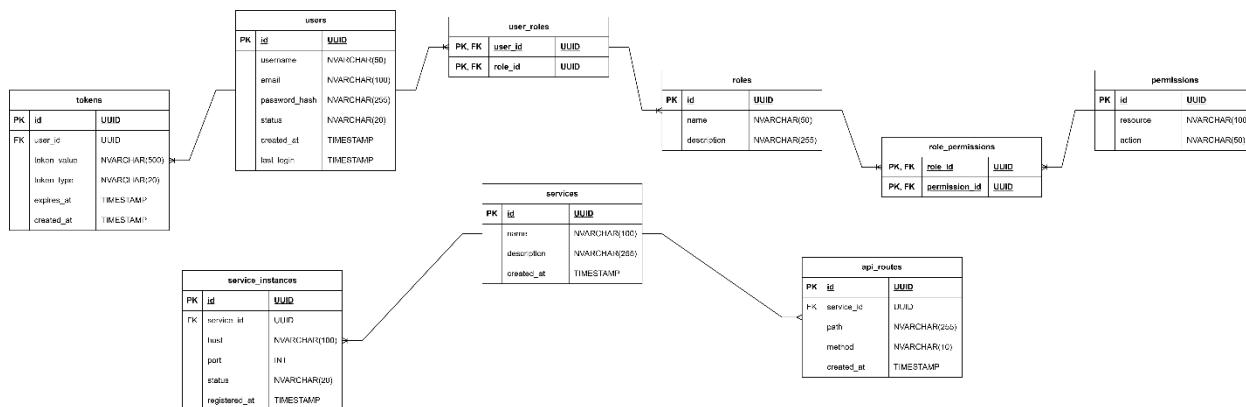


Рисунок 11 – Физическая модель базы данных

2.2.9 Разработка диаграммы размещения

Диаграмма размещения описывает физическое распределение компонентов системы по узлам инфраструктуры и показывает, как программные модули развёрнуты и взаимодействуют между собой. Диаграмма размещения представлена на рисунке 12.

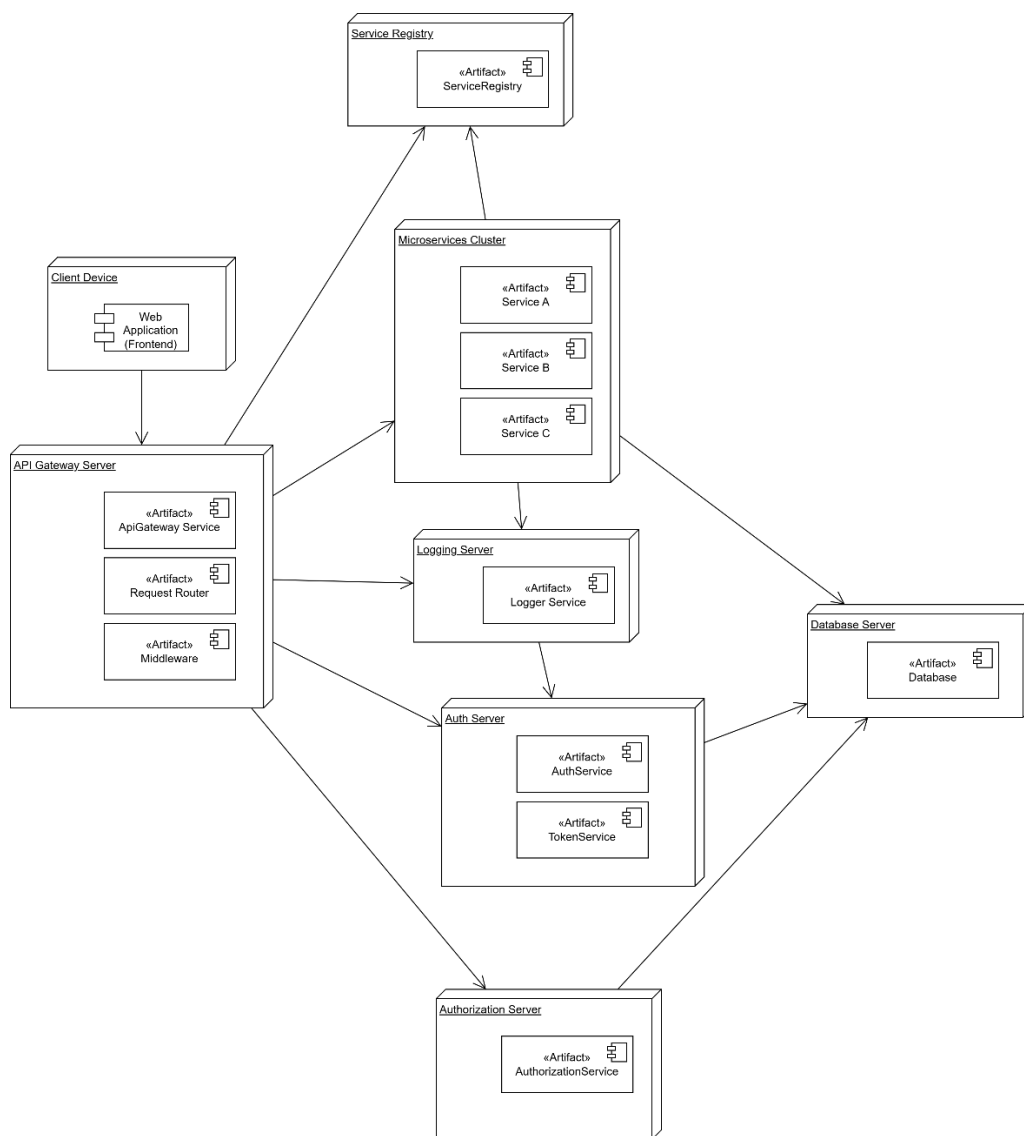


Рисунок 12 – Диаграмма размещения

2.3 Проектирование программного модуля

2.3.1 Разработка алгоритмов реализации основных функций программного обеспечения

Процесс работы программного обеспечения начинается с запуска ядра цифровой экосистемы и инициализации его основных компонентов: API-шлюза, сервиса аутентификации, сервиса авторизации, реестра микросервисов и базы данных. После запуска система проверяет доступность подключённых сервисов и готовность инфраструктуры к обработке запросов.

На первом этапе пользователь обращается к системе через клиентское приложение. Все входящие запросы направляются не напрямую к микросервисам, а через API-шлюз, который является единой точкой входа. Если пользователь выполняет вход в систему, API-шлюз передаёт учетные данные в сервис аутентификации. Сервис проверяет логин и пароль, после чего при успешной проверке формирует токены доступа.

Следующим этапом является проверка прав пользователя. При обращении к защищённому ресурсу API-шлюз извлекает токен из запроса и передаёт его на проверку в сервис аутентификации. После подтверждения подлинности пользователя выполняется обращение к сервису авторизации, где определяется роль пользователя и проверяется наличие необходимых прав доступа к запрашиваемому ресурсу.

Если пользователь обладает необходимыми правами, API-шлюз определяет, какой микросервис должен обработать запрос. Для этого он обращается к реестру сервисов, получает адрес нужного микросервиса и перенаправляет запрос. Микросервис выполняет требуемую операцию и возвращает результат обратно через API-шлюз.

При отсутствии токена, его недействительности или недостатке прав доступа система формирует соответствующий ответ с ошибкой: например, отказ в аутентификации или запрет доступа. Это позволяет централизованно контролировать безопасность и не допускать прямого обращения к внутренним сервисам.

Отдельный алгоритм предусмотрен для администрирования системы. Администратор может управлять учетными записями пользователей, ролями, правами доступа и зарегистрированными микросервисами. Все изменения сохраняются в базе данных и учитываются при последующих проверках доступа.

Также в системе реализуется алгоритм регистрации и подключения новых микросервисов. Новый сервис передаёт сведения о себе в реестр сервисов: название, адрес, доступные маршруты и статус. После регистрации

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						26
Изм.	Лист	№ докум.	Подпись	Дата		

API-шлюз может использовать эти сведения для маршрутизации запросов без изменения основной логики ядра системы.

Все действия пользователей и сервисов фиксируются в журнале событий. Логирование позволяет отслеживать процесс обработки запросов, выявлять ошибки, контролировать доступ и анализировать работу системы. Дополнительно выполняется мониторинг состояния микросервисов, что позволяет своевременно определять недоступные компоненты.

2.3.2 Проектирование пользовательского интерфейса

Поскольку разрабатываемая система представляет собой ядро цифровой экосистемы на основе микросервисной архитектуры, основной акцент в проекте сделан на серверной части, механизмах маршрутизации, аутентификации, авторизации и взаимодействия между сервисами. По этой причине полноценный пользовательский интерфейс не является центральным элементом разрабатываемого программного продукта.

Тем не менее для практической демонстрации работоспособности архитектуры был спроектирован минимальный пользовательский интерфейс демонстрационного микросервиса. Его назначение заключается не в реализации законченного клиентского приложения, а в обеспечении наглядного взаимодействия с ключевыми функциями ядра системы в процессе тестирования и демонстрации. Интерфейс программного обеспечения представлен на рисунке 13 в приложении Б.

Разработанный интерфейс решает следующие задачи:

- регистрация пользователя в системе;
- выполнение входа и получение токена доступа;
- создание роли и назначение её пользователю;
- создание permission и назначение его роли;
- вызов публичных и защищённых endpoint через API Gateway;
- отображение отправленного запроса и полученного ответа.

С точки зрения проектирования интерфейс построен по принципу минимализма и функциональной наглядности. Основной целью являлось обеспечить простое и быстрое выполнение сценариев демонстрации без перегрузки пользователя лишними элементами. В связи с этим структура интерфейса была организована в виде одной страницы, разделённой на несколько логических областей.

В верхней части страницы располагается заголовок, идентифицирующий демонстрационную панель. Левая часть интерфейса содержит блок текущего состояния системы, в котором отображаются ключевые данные текущей сессии: идентификатор пользователя, идентификатор роли, идентификатор permission и токен доступа. Это позволяет визуально отслеживать результаты выполненных операций и использовать полученные значения в последующих шагах сценария.

Центральная часть страницы содержит основные элементы управления, сгруппированные по функциональному назначению. В данном блоке пользователь может выполнить регистрацию, вход, создать роль, назначить роль, создать permission, назначить permission, а также отправить запросы к демонстрационным маршрутам через API Gateway. Такое разбиение соответствует логике бизнес-сценария и упрощает демонстрацию последовательности работы микросервисов.

Правая часть страницы предназначена для отображения обмена данными с системой. В одной области показывается отправляемый JSON-запрос, в другой – полученный ответ сервиса. Данное решение особенно важно для дипломной демонстрации, поскольку позволяет одновременно показать и пользовательское действие, и его техническую реализацию на уровне API-взаимодействия.

При проектировании интерфейса учитывались следующие требования:

- простота освоения и использования;
- наглядность последовательности действий;
- соответствие демонстрационным сценариям дипломного проекта;

- отсутствие избыточных элементов;
- удобство визуального сопоставления запроса и ответа.

Таким образом, пользовательский интерфейс в рамках данного проекта выполняет вспомогательную, демонстрационную функцию. Он подтверждает работоспособность разработанного ядра цифровой экосистемы, позволяет наглядно представить взаимодействие между микросервисами и повышает удобство практической демонстрации результатов разработки.

2.4 Реализация программного обеспечения на выбранном языке программирования и в выбранной среде разработки

Для реализации программного обеспечения был выбран язык программирования Python. Данный выбор обусловлен его широким применением в области веб-разработки, наличием развитой экосистемы библиотек и фреймворков, а также удобством создания серверных приложений с микросервисной архитектурой. Python обеспечивает высокую скорость разработки, хорошую читаемость исходного кода и позволяет удобно структурировать проект, что особенно важно при создании дипломного программного продукта.

В качестве основного веб-фреймворка был выбран FastAPI. Данный фреймворк предоставляет средства для быстрой разработки HTTP API, встроенную валидацию входных и выходных данных, автоматическую генерацию документации и удобную интеграцию с современными инструментами backend-разработки. Использование FastAPI позволило реализовать взаимодействие между микросервисами через REST API и обеспечить единообразный подход к обработке запросов.

Для работы с базой данных использовалась библиотека SQLAlchemy. Она была выбрана как ORM-инструмент, позволяющий описывать сущности предметной области в виде классов языка Python и отделять бизнес-логику приложения от низкоуровневых SQL-запросов. В качестве системы управления базами данных использовалась PostgreSQL. Данная СУБД

					ДП(Р).09.02.07 ПЗ	Лист
						29
Изм.	Лист	№ докум.	Подпись	Дата		

отличается надёжностью, поддержкой транзакций и хорошо подходит для хранения структурированных данных микросервисной системы.

Для хранения вспомогательных данных и поддержки отдельных сценариев был выбран Redis. Контейнеризация приложения выполнялась с использованием Docker и Docker Compose, что позволило организовать единое окружение запуска для всех сервисов и обеспечить удобное развёртывание проекта. Для запуска ASGI-приложений использовался сервер Uvicorn [11].

Разработка проекта велась в среде Visual Studio Code. Данная среда разработки была выбрана благодаря удобной работе с Python-проектами, поддержке структуры монорепозитория, встроенным средствам навигации по исходному коду и интеграции с системой контроля версий Git. Для управления версиями исходного кода использовался Git, а для хранения удалённого репозитория – GitHub.

Реализация программного обеспечения выполнялась поэтапно.

На первом этапе была сформирована общая структура проекта в виде монорепозитория. Внутри репозитория были выделены отдельные каталоги для каждого микросервиса: API Gateway, Auth Service, Authorization Service, User Service, Service Registry и Demo Microservice. Кроме того, был создан общий модуль shared, содержащий повторно используемые компоненты: конфигурацию, схемы ответов, инструменты безопасности, исключения и вспомогательные утилиты.

На втором этапе была настроена базовая инфраструктура проекта. Для этого был создан файл docker-compose.yml, в котором были описаны контейнеры PostgreSQL, Redis и всех микросервисов. Также были созданы Dockerfile для каждого сервиса и файл переменных окружения .env, обеспечивающий централизованную настройку параметров подключения и конфигурации.

На третьем этапе были реализованы общие механизмы системы. К ним относятся:

— централизованная загрузка конфигурации;

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						30
Изм.	Лист	№ докум.	Подпись	Дата		

- подключение к базе данных;
- единый формат успешных и ошибочных API-ответов;
- базовые пользовательские исключения;
- механизмы хэширования паролей;
- генерация и декодирование JWT-токенов.

На четвёртом этапе был реализован сервис управления пользователями (`user_service`). В нём были созданы модель пользователя, слой репозитория для работы с данными, слой сервисной логики и набор HTTP endpoint для выполнения операций создания, чтения, изменения, удаления, блокировки и разблокировки пользователей.

На пятом этапе был реализован сервис аутентификации (`auth_service`). В его состав вошли функции регистрации пользователя, входа в систему, генерации access и refresh token, обновления токенов, выхода из системы и смены пароля. Для хранения refresh token была использована отдельная сущность токена в базе данных.

На шестом этапе был реализован сервис авторизации (`authorization_service`). В рамках данного сервиса были созданы модели ролей, permissions и промежуточных таблиц связей. Реализованы операции создания роли, создания permission, назначения роли пользователю, назначения permission роли, а также проверки доступа пользователя к ресурсу по схеме role-based access control.

На седьмом этапе был реализован сервис реестра (`service_registry`). Он обеспечивает регистрацию микросервисов, хранение информации об их маршрутах, базовых адресах и статусах. Это позволило организовать расширяемую архитектуру, в которой новый сервис может быть подключён без изменения логики ядра. Ниже представлен код, реализующий работу `service_registry`:

```
from service_registry.app.models.api_route import ApiRoute
from service_registry.app.models.service import ServiceEntity
from service_registry.app.repositories.service_repository import ServiceRepository
```

					ДП(Р).09.02.07 ПЗ	Лист
						31
Изм.	Лист	№ докум.	Подпись	Дата		

```

from service_registry.app.schemas.registry import ServiceRegisterRequest
from shared.exceptions.api import NotFoundException

class ServiceRegistryService:
    """Registry for microservice metadata and routes."""
    def __init__(self, repository: ServiceRepository) -> None:
        self.repository = repository
    def register_service(self, data: ServiceRegisterRequest) -> ServiceEntity:
        service = self.repository.get_service_by_name(data.name)
        if service:
            service.base_url = data.base_url
            service.health_url = data.health_url
            service.status = data.status
            service.version = data.version
            service = self.repository.save_service(service)
        else:
            service = self.repository.create_service(
                ServiceEntity(
                    name=data.name,
                    base_url=data.base_url,
                    health_url=data.health_url,
                    status=data.status,
                    version=data.version,
                )
            )
        for route in data.routes:
            self.repository.create_route(
                ApiRoute(
                    service_id=service.id,
                    path=route.path,
                    method=route.method.upper(),

```

					ДП(Р).09.02.07 ПЗ	Лист
						32
Изм.	Лист	№ докум.	Подпись	Дата		


```

        is_protected=route.is_protected,
        required_permission=route.required_permission,
    )
)

return service

def remove_service(self, service_id: str) -> None:
    service = self.repository.get_service_by_id(service_id)
    if not service:
        raise NotFoundException("Service not found")
    self.repository.delete_service(service)

def get_service_by_name(self, name: str) -> ServiceEntity:
    service = self.repository.get_service_by_name(name)
    if not service:
        raise NotFoundException("Service not found")
    return service

def list_services(self) -> list[ServiceEntity]:
    return self.repository.list_services()

def update_status(self, service_id: str, status: str) -> ServiceEntity:
    service = self.repository.get_service_by_id(service_id)
    if not service:
        raise NotFoundException("Service not found")
    service.status = status
    return self.repository.save_service(service)

def resolve_route(self, path: str, method: str) -> tuple[ServiceEntity, ApiRoute]:
    normalized_method = method.upper()
    for route in self.repository.list_all_routes():
        if route.path == path and route.method == normalized_method:
            service = self.repository.get_service_by_id(route.service_id)
            if not service:

```

```
break  
return service, route  
raise NotFoundException("Route not found")
```

На восьмом этапе был реализован API Gateway. В нём были настроены механизмы приёма внешних запросов, валидации токена через Auth Service, проверки доступа через Authorization Service, поиска маршрута через Service Registry и последующего проксирования запроса в нужный микросервис. Дополнительно были реализованы базовое логирование запросов и ограничение частоты обращений.

На девятом этапе был разработан демонстрационный микросервис (demo_microservice). Его назначение состоит в подтверждении работоспособности ядра экосистемы. Внутри него были созданы публичный маршрут, защищённый маршрут и административный маршрут, доступ к которому предоставляется только при наличии соответствующего permission.

На заключительном этапе был реализован минимальный демонстрационный пользовательский интерфейс. Он предназначен для наглядного показа ключевых сценариев работы системы: регистрации, входа, создания роли, назначения permission и вызова маршрутов через API Gateway. Наличие данного интерфейса упростило тестирование и сделало демонстрацию проекта более наглядной.

2.5 Выбор стратегии тестирования, разработка тестов, тестирование и отладка программного обеспечения

Для обеспечения корректности работы разработанного программного обеспечения была выбрана комбинированная стратегия тестирования, включающая модульное и функциональное тестирование. Такой подход позволяет проверить как отдельные программные компоненты, так и основные пользовательские сценарии взаимодействия между сервисами.

На этапе проектирования тестирования учитывалось, что разрабатываемая система представляет собой микросервисное ядро цифровой

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						34
Изм.	Лист	№ докум.	Подпись	Дата		

экосистемы. В связи с этим особое внимание было уделено проверке наиболее значимых с точки зрения архитектуры подсистем: аутентификации, авторизации, маршрутизации запросов, работы реестра сервисов и демонстрационного микросервиса.

В качестве основного инструмента тестирования был выбран фреймворк `pytest`. Данный инструмент был выбран благодаря простоте написания тестов, удобной структуре фикстур, поддержке асинхронных сценариев и широкому распространению в Python-разработке. Для проверки HTTP-маршрутов демонстрационного микросервиса использовался `TestClient`, а для проверки сервисной логики применялись модульные тесты с изолированными заглушками репозитория.

В ходе разработки были созданы тесты следующих категорий:

- тесты сервиса аутентификации;
- тесты сервиса авторизации;
- тесты сервиса реестра;
- тесты логики API Gateway;
- тесты маршрутов демонстрационного микросервиса.

Для `auth_service` были разработаны тесты, проверяющие успешную регистрацию пользователя, отказ при повторной регистрации с тем же именем, отказ при неверном пароле, обновление `access token` по `refresh token`, а также отзыв `refresh token`. Эти тесты подтверждают корректность базовых сценариев аутентификации и управления токенами.

Для `authorization_service` были разработаны тесты, проверяющие создание ролей и `permissions`, повторное создание уже существующих сущностей, назначение ролей и `permissions`, а также проверку доступа пользователя к ресурсу. Дополнительно были предусмотрены негативные сценарии, связанные с отсутствием обязательных идентификаторов и обращением к несуществующей роли.

Для `service_registry` были реализованы тесты регистрации сервиса и маршрута, а также тест разрешения маршрута по пути и HTTP-методу. Это

					ДП(Р).09.02.07 ПЗ	Лист
						35
Изм.	Лист	№ докум.	Подпись	Дата		

позволило проверить корректность логики, на основе которой API Gateway определяет конечный сервис для проксирования запроса.

Для api_gateway были разработаны тесты, проверяющие отказ в доступе при отсутствии bearer token и отказ при отрицательном результате проверки прав. Данные тесты подтверждают корректность базовой защитной логики шлюза.

Для demo_microservice были реализованы функциональные тесты маршрутов public, private и admin. В рамках этих тестов были проверены как позитивные, так и негативные сценарии: успешное получение публичного ответа, отказ при отсутствии пользовательского контекста, отказ при отсутствии роли администратора и успешный доступ при наличии роли ADMIN. Ниже представлен код тестов для demo_microservice, проверяющих маршруты:

```
from fastapi.testclient import TestClient
from demo_microservice.app.main import app
client = TestClient(app)

def test_public_endpoint_returns_success() -> None:
    response = client.get("/demo/public")
    assert response.status_code == 200
    assert response.json()["success"] is True
    assert response.json()["data"]["scope"] == "public"

def test_private_endpoint_requires_user_header() -> None:
    response = client.get("/demo/private")
    assert response.status_code == 401
    assert response.json()["detail"] == "Missing user context"

def test_admin_endpoint_rejects_non_admin_role() -> None:
    response = client.get("/demo/admin", headers={"x-user-id": "user-1", "x-user-roles": "USER"})
    assert response.status_code == 403
    assert response.json()["detail"] == "Admin role required"
```

					ДП(Р).09.02.07 ПЗ	Лист
						36
Изм.	Лист	№ докум.	Подпись	Дата		

```
def test_admin_endpoint_accepts_admin_role() -> None:
    response = client.get("/demo/admin", headers={"x-user-id": "user-1", "x-user-roles": "ADMIN"})
    assert response.status_code == 200
    assert response.json()["data"]["scope"] == "admin"
```

Разработанные тесты позволили не только проверить корректность работы системы, но и выявить отдельные ошибки в процессе отладки. В ходе тестирования и ручной проверки были обнаружены и устранены следующие проблемы:

- конфликт при генерации токенов при повторных быстрых логинах;
- ошибки при повторном создании ролей и permissions;
- ошибки, связанные с обработкой пустых идентификаторов роли и permission;
- отдельные проблемы пользовательского интерфейса демонстрационного микросервиса.

Для первичной технической проверки корректности тестов и исходного кода дополнительно использовалась синтаксическая компиляция Python-модулей. Это позволило убедиться в отсутствии синтаксических ошибок в тестах и основных программных компонентах.

3 Экономическая часть

Экономическое обоснование разработки ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом необходимо для оценки целесообразности создания собственного программного решения по сравнению с использованием готовых корпоративных платформ.

Разрабатываемая система представляет собой серверное программное обеспечение, предназначенное для централизованного управления аутентификацией пользователей, маршрутизации запросов и организации взаимодействия между микросервисами. В процессе разработки используются современные технологии серверной разработки, контейнеризации и взаимодействия распределённых сервисов.

Исходные данные для проведения экономических расчётов представлены в таблице 1.

Таблица 1 – Исходные данные для экономических расчётов

Параметр	Значение
Режим работы	5 дней в неделю
Продолжительность рабочего дня	8 часов
Продолжительность разработки	2 месяца
Количество рабочих дней	42 дня
Общая трудоёмкость разработки	336 часов
Средняя стоимость часа разработки	1500 руб./час
Отчисления на социальные нужды	30 %
Стоимость VPS-сервера	4000 руб.
Стоимость домена и SSL-сертификата	1000 руб.

Общая трудоёмкость разработки определяется по формуле:

$$T = D \cdot H, \quad (3.1)$$

где:

T – общая трудоёмкость разработки, ч;

D – количество рабочих дней;

H – продолжительность рабочего дня, ч.

Подставим значения в формулу (3.1):

$$T = 42 \cdot 8 = 336 \text{ ч.}$$

Таким образом, суммарная трудоёмкость разработки программного обеспечения составляет 336 часов.

Расчёт стоимости разработки программного обеспечения

Разработка ядра цифровой экосистемы требует выполнения задач по проектированию микросервисной архитектуры, реализации API-шлюза, настройки системы аутентификации и авторизации, а также организации взаимодействия между сервисами.

Основную часть затрат составляют расходы на оплату труда разработчика.

Затраты на разработку определяются по формуле:

$$Z_{\text{разр}} = T \cdot C_{\text{час}}, \quad (3.2)$$

где:

$Z_{\text{разр}}$ – затраты на оплату труда разработчика, руб.;

T – трудоёмкость разработки, ч;

$C_{\text{час}}$ – стоимость одного часа разработки, руб./ч.

Подставим значения в формулу (3.2):

$$Z_{\text{разр}} = 336 \cdot 1500 = 504\,000 \text{ руб.}$$

Дополнительно учитываются отчисления на социальные нужды, рассчитываемые по формуле:

$$O_{\text{соц}} = Z_{\text{разр}} \cdot 0,3, \quad (3.3)$$

где:

$O_{\text{соц}}$ – отчисления на социальные нужды, руб.

Подставим значения в формулу (3.3):

$$O_{\text{соц}} = 504\,000 \cdot 0,3 = 151\,200 \text{ руб.}$$

Полные затраты на оплату труда определяются по формуле:

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						39
Изм.	Лист	№ докум.	Подпись	Дата		

$$Z_{\text{общ}} = Z_{\text{разр}} + O_{\text{соц}}, \quad (3.4)$$

где:

$Z_{\text{общ}}$ – общие затраты на разработку с учётом социальных отчислений.

Подставим значения в формулу (3.4):

$$Z_{\text{общ}} = 504\,000 + 151\,200 = 655\,200 \text{ руб.}$$

Результаты расчётов представлены в таблице 2.

Таблица 2 – Расчёт затрат на разработку

Параметр	Значение
Продолжительность разработки	2 месяца
Рабочих дней	42 дня
Часов в день	8
Общая трудоёмкость	336 часов
Стоимость 1 часа разработки	1500 руб.
Затраты на оплату труда разработчика	504 000 руб.
Отчисления на социальные нужды	151 200 руб.
Общие затраты на разработку	655 200 руб.

Капитальные затраты на разработку

Помимо затрат на оплату труда, для разработки программного обеспечения необходимы дополнительные расходы, связанные с использованием серверной инфраструктуры и обеспечением безопасного доступа к системе.

Капитальные затраты определяются по формуле:

$$K = Z_{\text{разр}} + C_{VPS} + C_{SSL}, \quad (3.5)$$

где:

K – капитальные затраты на разработку, руб.;

$Z_{\text{разр}}$ – затраты на оплату труда разработчика, руб.;

C_{VPS} – стоимость аренды VPS-сервера, руб.;

C_{SSL} – стоимость домена и SSL-сертификата, руб.

Подставим значения в формулу (3.5):

$$K = 504\,000 + 4\,000 + 1\,000 = 509\,000 \text{ руб.}$$

Результаты представлены в таблице 3.

Таблица 3 – Капитальные затраты

Статья затрат	Сумма, руб.
Разработка программного обеспечения	504 000
VPS-сервер на период разработки	4 000
Домен и SSL-сертификат	1 000
Итого капитальные затраты	509 000

Оборудование разработчика (персональный компьютер, монитор и периферийные устройства) в расчётах не учитывается, так как использовалось личное оборудование.

Сравнение с существующими аналогами

Для оценки экономической эффективности разработанного решения проведём сравнение с существующими платформами, обеспечивающими централизованную аутентификацию, управление API и организацию взаимодействия между сервисами.

В качестве аналогов выбраны следующие решения:

- Keycloak + Kong Gateway – популярное решение для организации единой аутентификации и API-шлюза.
- Auth0 + API Gateway – облачная платформа для управления доступом и API.
- Разрабатываемое программное обеспечение.

Таблица 4 – Сравнение стоимости существующих решений

Решение	Стоимость внедрения	Годовое сопровождение
Keycloak + Kong Gateway	300 000 руб.	70 000 руб.
Auth0 + API Gateway	450 000 руб.	120 000 руб.
Разрабатываемая система	509 000 руб.	20 000 руб.

Несмотря на то, что собственная разработка требует больших первоначальных вложений, она позволяет избежать регулярных расходов на лицензирование и сопровождение сторонних платформ. Кроме того, собственное решение может быть адаптировано под конкретные требования организации и расширено без ограничений, связанных с использованием коммерческих продуктов.

Расчёт экономического эффекта

Экономический эффект от внедрения системы заключается в:

- снижении затрат на сопровождение нескольких разрозненных сервисов;
- уменьшении времени на интеграцию новых микросервисов;
- упрощении управления пользователями и правами доступа;
- снижении нагрузки на администраторов инфраструктуры;
- уменьшении затрат на поддержку отдельных систем аутентификации.

Предположим, что использование единого ядра цифровой экосистемы позволяет экономить организации 300 000 руб. в год за счёт сокращения времени сопровождения инфраструктуры и уменьшения административных затрат.

За три года экономия составит:

$$\mathcal{E}_{3г} = \mathcal{E}_{год} \cdot 3, \quad (3.6)$$

где:

$\mathcal{E}_{3г}$ – экономический эффект за 3 года;

$\mathcal{E}_{год}$ – годовая экономия.

Подставим значения в формулу (3.6):

$$\mathcal{E}_{3г} = 300\,000 \cdot 3 = 900\,000 \text{ руб.}$$

Чистый экономический эффект определяется как разница между полученной экономией и дополнительными затратами на разработку:

$$\mathcal{E}_{чист} = \mathcal{E}_{3г} - (K - C_{аналога}), \quad (3.7)$$

где:

$\mathcal{E}_{чист}$ – чистый экономический эффект;

К – капитальные затраты на разработку;

$C_{\text{аналога}}$ – стоимость внедрения готового аналога.

В качестве аналога принимается решение Keycloak + Kong Gateway стоимостью 300 000 руб.

Подставим значения в формулу (3.7):

$$\mathcal{E}_{\text{чист}} = 900\,000 - (509\,000 - 300\,000) = 691\,000 \text{ руб.}$$

Полученные результаты показывают, что внедрение собственного ядра цифровой экосистемы является экономически целесообразным и обеспечивает положительный экономический эффект в долгосрочной перспективе.

Расчёт срока окупаемости

Срок окупаемости определяется как отношение капитальных затрат к ежегодной экономии:

$$PP = \frac{K}{\mathcal{E}_{\text{год}}}, \quad (3.8)$$

где:

PP – срок окупаемости, лет;

К – капитальные затраты, руб.;

$\mathcal{E}_{\text{год}}$ – ежегодный экономический эффект, руб.

Подставим значения в формулу (3.8):

$$PP = \frac{509\,000}{300\,000} \approx 1,7 \text{ года}$$

Таким образом, срок окупаемости разработанного программного обеспечения составляет приблизительно 1,7 года, или около 20,4 месяцев.

Проведённые расчёты показывают, что разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом является экономически обоснованной. Несмотря на капитальные затраты в размере 509 000 руб., система обеспечивает положительный экономический эффект 691 000 руб. за 3 года и окупается менее чем за два года, что подтверждает целесообразность её внедрения в корпоративную инфраструктуру.

					ДП(Р).09.02.07 ПЗ	Лист
						43
Изм.	Лист	№ докум.	Подпись	Дата		

Охрана труда

Общие требования охраны труда:

1. К работе программистом допускаются лица не моложе 18 лет, имеющие соответствующую выполняемой работе квалификацию, прошедшие вводный и первичный на рабочем месте инструктажи по охране труда, медосмотр, обучение и проверку знаний по охране труда.

2. Для выполнения работ на персональном компьютере программист должен изучить инструкцию по эксплуатации персонального компьютера, на котором работник выполняет работы, пройти инструктаж по электробезопасности и получить I группу.

3. Программист, выполняющий работу на персональном компьютере, независимо от квалификации и стажа работы не реже одного раза в шесть месяцев должен проходить повторный инструктаж по безопасности труда. в случае нарушения требований безопасности труда, при перерыве в работе более чем на 60 календарных дней программист должен пройти внеплановый инструктаж.

4. Программист, показавший неудовлетворительные навыки и знания требований безопасности при работе на персональном компьютере, к самостоятельной работе не допускается.

5. Программист, допущенный к самостоятельной работе, должен знать:

5.1. Правила эксплуатации и требования безопасности при работе с персональным компьютером.

5.2. Способы рациональной организации рабочего места.

5.3. Санитарно-гигиенические требования к условиям труда.

5.4. Опасные и вредные производственные факторы, которые могут оказывать неблагоприятное воздействие на программиста.

6. Программист, направленный для участия в несвойственных его профессии работах, должен пройти целевой инструктаж по безопасному выполнению предстоящих работ.

					ДП(Р).09.02.07 ПЗ	Лист
						44
Изм.	Лист	№ докум.	Подпись	Дата		

7. Программист, работающий на персональном компьютере, должен соблюдать установленные для него режимы труда и отдыха.

8. Для предупреждения возможности возникновения пожара программист должен соблюдать требования пожарной безопасности сам и не допускать нарушений со стороны других работников.

9. Для предупреждения заболеваний программисту следует знать и соблюдать правила личной гигиены.

10. В случае заболевания, плохого самочувствия, недостаточного отдыха программисту следует сообщить о своем состоянии непосредственному руководителю и обратиться за медицинской помощью.

11. Если программист оказался очевидцем несчастного случая, он должен оказать пострадавшему первую помощь и сообщить о случившемся руководителю.

12. Программист должен уметь оказать первую помощь, в том числе при поражении электрическим током, пользоваться аптечкой.

13. Программист, допустивший нарушение или невыполнение требований инструкции по охране труда, несет ответственность согласно действующему законодательству.

Опасные и вредные производственные факторы:

1. Перенапряжение зрительного анализатора при работе за экраном дисплея.

2. Длительное статическое напряжение мышц спины, шеи, рук и ног, что может привести к статическим перегрузкам программиста.

3. Повышенный уровень шума.

4. Ионизирующие и неионизирующие излучения, источниками которых являются видеодисплейные терминалы.

5. Статическое электричество.

6. Электрический ток, путь которого в случае замыкания на корпус может пройти через тело человека.

Рабочие помещения и рабочие места оператора ПК должны соответствовать определенным требованиям, которые обеспечат комфортную и безопасную работу. К ним относятся:

1. Размеры помещения не менее 6 квадратных метров на одно рабочее место.
2. Наличие естественного и искусственного освещения, соответствующего СанПиН 2.2.2./2.4.134003 «Гигиенические требования к условиям труда при работе на персональных компьютерах».
3. Температура воздуха в помещении должна находиться в пределах 18-24 градусов Цельсия, влажность 40-60%.
4. Отсутствие шума и других нежелательных звуковых эффектов.
5. Наличие пожарных средств и путей эвакуации.
6. Наличие мебели, соответствующей правилам эргономики (регулируемая высота, регулируемый наклон стола, спинка и сиденье стула, наличие подставки для ног и т.д.).
7. Наличие специального оборудования для защиты здоровья (экраны, фильтры, подставки для документов и т.д.).
8. Соответствие параметров компьютера установленным нормам.
9. Работа оператора ПК должна проходить в условиях микроклимата, обеспечивающего наиболее комфортные условия для зрения (уровень освещенности должен быть не менее 300 лк).
10. Обеспечение необходимой пространственной ориентации с помощью различных методов размещения монитора и клавиатуры.

Общие требования при использовании компьютерной техники:

1. Защита зрения.
 - 1.1. Правильное расстояние до экрана (60-70 см), регулировка яркости и контрастности, отсутствие бликов.
 - 1.2. Использование щадящих цветовых схем (светло-зеленые, желто-зеленые тона), темные символы на светлом фоне.

1.3. Каждые 20-30 минут отводить взгляд от экрана, фокусироваться на удаленных объектах (правило “202020”: каждые 20 минут смотреть на объект на расстоянии 20 футов (около 6 метров) в течение 20 секунд).

1.4. Выполнение простых упражнений для расслабления мышц глаз (например, перемещение взгляда вверх-вниз, вправо-влево, по кругу).

1.5. Использование увлажняющих капель (“искусственная слеза”) при сухости глаз.

1.6. Правильное освещение рабочего места (не слишком яркое, не слишком тусклое, без бликов), использование настольной лампы.

1.7. Использование программ, фильтрующих синий свет, изменяющих цветовую температуру экрана в зависимости от времени суток.

1.8. Регулярные осмотры у офтальмолога (не реже одного раза в год).

1.9. Использование мониторов с матовым покрытием или антибликовыми пленками.

1.10. Настройка параметров шрифта для комфортного чтения.

2. Защита органов слуха.

2.1. Использование тихо работающих компьютеров и периферийных устройств (вентиляторов, жестких дисков).

2.2. Обеспечение звукоизоляции рабочего места (например, использование звукопоглощающих панелей).

2.3. Использование наушников или гарнитуры для телефонных разговоров, что позволяет снизить уровень окружающего шума. При этом громкость звука в наушниках должна быть умеренной.

2.4. Регулярные перерывы в тишине, отдых от звуковых раздражителей.

2.5. Использование берушей при работе в шумной обстановке.

2.6. Регулярные осмотры у отоларинголога (особенно при наличии факторов риска, таких как работа в шумной среде).

3. Защита органов пищеварения.

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						47
Изм.	Лист	№ докум.	Подпись	Дата		

3.1. Соблюдение режима питания (завтрак, обед, ужин), прием пищи в одно и то же время.

3.2. Употребление достаточного количества овощей, фруктов, цельнозерновых продуктов, нежирного белка. Ограничение потребления жирной, жареной, сладкой пищи, фастфуда.

3.3. Употребление достаточного количества воды (не менее 1,52 литров в день).

3.4. Выбор полезных перекусов (фрукты, овощи, орехи, йогурт).

3.5. Использование обеденного перерыва для полноценного приема пищи, отдыха от работы.

3.6. Рекомендуется есть в специально отведенных для этого местах.

3.7. Соблюдать режим питания, не переедать перед сном.

3.8. Тщательно мыть руки перед едой.

4. Защита кожных покровов.

4.1. Использование увлажнителей воздуха в помещении.

4.2. Регулярное проветривание помещения.

4.3. Умывание лица несколько раз в день.

4.4. Использование увлажняющих кремов для рук и лица.

4.5. Использование минимального количества косметики, выбор гипоаллергенной косметики.

4.6. Хотя прямое воздействие электромагнитного излучения компьютеров на кожу незначительно, рекомендуется соблюдать дистанцию от экрана и других устройств.

4.7. Регулярная уборка рабочего места, очистка клавиатуры и мыши от пыли и грязи.

4.8. Использование антистатических ковриков.

5. Установка расписания работы и время отдыха в течение дня.

5.1. Соблюдение режима труда и отдыха, установленного нормативными документами. Продолжительность непрерывной работы с видеомонитором не должна превышать 2 часов.

					ДП(Р).09.02.07 ПЗ	Лист
						48
Изм.	Лист	№ докум.	Подпись	Дата		

5.2. Короткие перерывы (1-3 минуты) каждые 20-30 минут для снятия напряжения.

5.3. Выполнение простых физических упражнений во время перерывов для улучшения кровообращения и снятия мышечного напряжения.

5.4. Чередование различных видов деятельности, планирование задач с учетом пиков и спадов работоспособности.

5.5. Достаточный сон (78 часов) для восстановления сил.

5.6. Полноценный отдых в выходные дни.

5.7. Регулярный отпуск для восстановления физического и психического здоровья.

5.8. Правильная организация рабочего места (удобное кресло, регулируемый стол, правильное расположение монитора и клавиатуры) помогает снизить утомляемость и риск возникновения заболеваний.

Техника безопасности перед началом работы:

1. Перед началом работы программисту следует рационально организовать свое рабочее место.

2. Программист должен знать о том, что если в помещении расположены несколько персональных компьютеров, то для обеспечения безопасности расстояние между ними должно быть не менее 1,5 м.

3. Программист должен знать о том, что взаимное расположение персональных компьютеров влияет на уровень генерируемых ими излучений. для предупреждения облучения других рабочих мест следует выполнять следующие правила:

1.1. Левая панель персонального компьютера должна быть обращена либо к стене, либо к проходу, где нет рабочих мест.

1.2. Не следует располагать мониторы экранами друг к другу.

1.3. Не рекомендуется располагать монитор экраном к окну.

1.4. Для того, чтобы в процессе работы не возникало перенапряжение зрительного анализатора, программисту следует

проверить, чтобы на клавиатуре и экране монитора не было бликов света.

1.5. Для повышения контрастности изображения перед началом работы программист должен очистить экран монитора от пыли, которая интенсивно оседает на нем под воздействием зарядов статического электричества.

4. Программист должен убрать с рабочего места все лишние предметы, не используемые в работе.

5. Перед включением персонального компьютера программисту следует визуально проверить исправность электропроводки, вилки, розетки, а также электрических подсоединений между собой всех устройств, входящих в комплект персонального компьютера.

6. Перед началом выполнения работы программист должен проверить исправность персонального компьютера и подготовить его к работе.

Техника безопасности во время работы

1. Программисту персонального компьютера следует включать его в работу в той последовательности, которая определена инструкцией по эксплуатации.

2. Для подключения персонального компьютера к электрической сети программист должен использовать шнур питания, поставляемый в комплекте с персональным компьютером. не следует использовать самодельные электрические шнуры для подключения к сети персонального компьютера и различных его устройств.

3. Программист должен знать, что рациональная рабочая поза способствует уменьшению утомляемости.

4. При помощи поворотной площадки видеомонитор должен быть отрегулирован в соответствии с рабочей позой программиста.

5. Конструкция рабочего стула (кресла) должна обеспечивать поддержание рабочей позы программиста при работе с персональным

					ДП(Р).09.02.07 ПЗ	Лист
						50
Изм.	Лист	№ докум.	Подпись	Дата		

компьютером, позволять изменять позу с целью снижения статического напряжения мышц шейно-плечевой области и спины для предупреждения развития утомления.

6. Тип рабочего стула (кресла) должен выбираться в зависимости от характера и продолжительности работы с персональным компьютером с учетом роста программиста.

7. Рабочий стул (кресло) должен быть подъемно-поворотным и регулируемым по высоте и углам наклона сиденья и спинки, а также расстоянию спинки от переднего края сиденья. при этом регулировка каждого параметра должна быть независимой, легко осуществляемой и иметь надежную фиксацию.

8. Поверхность сиденья, спинки и других элементов стула (кресла) должна быть полумягкой, с нескользящим, неэлектризуемым и воздухопроницаемым покрытием, обеспечивающим легкую очистку от загрязнений.

9. Плоскость рабочего стола должна быть регулируемой по высоте в пределах 680–800 мм с учетом индивидуальных особенностей программиста. при отсутствии такой возможности высота рабочей поверхности стола должна составлять 725 мм.

10. Рабочий стол должен иметь пространство для ног высотой не менее 600 мм, шириной – не менее 500 мм, глубиной на уровне колен – не менее 450 мм и на уровне вытянутых ног – не менее 650 мм.

11. Экран видеомонитора должен находиться от глаз программиста на оптимальном расстоянии 600–700 мм, но не ближе 500 мм с учетом размеров алфавитно-цифровых знаков и символов.

12. Клавиатуру следует располагать на поверхности стола на расстоянии 100–300 мм от края, обращенного к пользователю, или на специальной, регулируемой по высоте рабочей поверхности, отделенной от основной столешницы.

13. Для уменьшения зрительной утомляемости программисту предпочтительнее работать в таком режиме, чтобы на светлом экране видеомонитора были темные символы.

14. С целью снижения зрительного и костно-мышечного утомления программисту следует соблюдать установленный режим труда и отдыха.

15. Режимы труда и отдыха при работе с персональным компьютером должны организовываться в зависимости от вида и категории трудовой деятельности.

Требования охраны труда в аварийных ситуациях

1. При обнаружении каких-либо неполадок в работе персонального компьютера программист должен прекратить работу, выключить компьютер и сообщить об этом непосредственному руководителю для организации ремонта.

2. Программисту не следует самому устранять технические неполадки персонального компьютера.

3. Программист не должен производить работу при снятом корпусе компьютера.

4. При несчастном случае, отравлении, внезапном заболевании необходимо немедленно оказать первую помощь пострадавшему, вызвать врача или помочь доставить пострадавшего к врачу, а затем сообщить руководителю о случившемся.

5. Программист должен уметь оказывать первую помощь при ранениях. при этом он должен знать, что всякая рана легко может загрязниться микробами, находящимися на ранящем предмете, коже пострадавшего, а также в пыли, на руках оказывающего помощь и на грязном перевязочном материале.

6. При обнаружении пожара или признаков горения (задымление, запах гари, повышение температуры и т. п.) необходимо немедленно уведомить об этом пожарную охрану по телефону 01.

7. До прибытия пожарной охраны нужно принять меры по эвакуации людей, имущества и приступить к тушению пожара.

8. При возгорании персонального компьютера программист должен отключить его от источника тока и приступить к тушению своими силами. при этом следует помнить, что для тушения установок, находящихся под напряжением, применяют углекислотные или порошковые огнетушители.

Техника безопасности по окончании работы

1. По окончании работы программист должен выключить персональный компьютер и отсоединить сетевой шнур от электрической сети.

2. Программист должен привести в порядок рабочее место, убрать документацию и т. п.

Современная жизнь невозможна без использования компьютеров и интернета. Тем не менее, работа на ПК может представлять опасность для здоровья человека, если не соблюдать правила безопасности. Такие заболевания, как синдром карпального канала, синдром "сухого глаза", головные боли и шум в ушах, связаны с длительным пребыванием за компьютером. В связи с этим, необходимо соблюдать следующие правила техники безопасности при работе на ПК:

1. Регулярно делать перерывы. При работе за компьютером необходимо делать перерыв каждые 45-60 минут. Во время перерывов рекомендуется выполнять упражнения для глаз и рук.

2. Правильно настроить рабочее место. Рабочее место должно быть правильно настроено, чтобы минимизировать риск различных заболеваний. Клавиатура должна быть расположена на уровне локтя, а экран на уровне глаз.

3. Использовать эргономическую мебель. Для работы за компьютером рекомендуется использовать эргономические кресла и столы, которые помогают поддерживать правильную позу.

4. Соблюдать правильный режим работы. Необходимо соблюдать правильный режим работы и сна, чтобы избежать усталости и стресса, которые могут привести к заболеваниям.

					ДП(Р).09.02.07 ПЗ	Лист
						53
Изм.	Лист	№ докум.	Подпись	Дата		

5. Использовать программы для защиты глаз. Существует множество программ, которые помогают защитить глаза при работе за компьютером. Они уменьшают яркость экрана, фильтруют синий свет и т.д.

6. Использовать антивирусное программное обеспечение. Антивирусное программное обеспечение помогает защитить компьютер от вирусов и злонамеренных программ, которые могут повредить систему или украсть личную информацию.

7. Избегать монотонной нагрузки. Повторяющиеся действия могут привести к снижению производительности и здоровью. Для этого рекомендуется использовать различные программы и ресурсы, менять виды деятельности, чтобы предотвратить нагрузку на конкретную группу мышц.

8. Использовать безопасные пароли. Для защиты личной информации и конфиденциальных данных следует использовать сложные пароли, которые не легко угадать или взломать. Рекомендуется использовать комбинации букв, цифр и символов.

9. Создавать резервные копии данных. Для сохранения важной информации необходимо регулярно создавать резервные копии данных, чтобы предотвратить потерю или повреждение важных файлов.

10. Обновлять программное обеспечение. Регулярное обновление программного обеспечения помогает устранять уязвимости и предотвращать атаки злонамеренных программ. Рекомендуется устанавливать обновления операционной системы, браузера и антивирусного ПО.

11. Использовать безопасную сеть. При работе на публичных сетях необходимо использовать VPN для защиты информации, передаваемой между компьютером и сервером или другим устройством.

12. Избегать использования нелегального программного обеспечения. Использование нелегального программного обеспечения может привести к нарушению законодательства и повышенному риску заражения вирусами и злонамеренными программами.

Заключение

В рамках выпускной квалификационной работы было спроектировано и реализовано ядро цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом. В ходе разработки был выполнен полный цикл создания программного обеспечения: от анализа предметной области и существующих технических решений до проектирования архитектуры системы, реализации программных компонентов, тестирования и экономического обоснования разработанного решения.

В процессе выполнения работы были решены следующие задачи:

- изучены современные подходы к построению микросервисных архитектур и цифровых платформ;
- проведён анализ предметной области и рассмотрены существующие решения в области организации единой аутентификации, управления доступом и маршрутизации запросов;
- разработана архитектура ядра цифровой экосистемы, основанная на принципах микросервисного взаимодействия;
- спроектированы основные программные компоненты системы, включающие сервис аутентификации, сервис авторизации, API-шлюз, реестр сервисов, систему логирования и мониторинга;
- разработана модель базы данных для хранения информации о пользователях, ролях, правах доступа, токенах и зарегистрированных сервисах [9];
- реализовано программное решение, обеспечивающее централизованную аутентификацию пользователей, управление доступом и маршрутизацию запросов между сервисами;
- проведено тестирование основных сценариев работы системы, включая аутентификацию пользователей, авторизацию, обработку запросов через API-шлюз и взаимодействие между микросервисами;

					<i>ДП(Р).09.02.07 ПЗ</i>	Лист
						55
Изм.	Лист	№ докум.	Подпись	Дата		

— выполнена экономическая оценка разработки и подтверждена целесообразность создания собственного программного решения.

Разработанная система обеспечивает централизованное управление пользователями и их правами доступа, безопасное взаимодействие между компонентами системы и единый механизм обработки запросов. Использование API-шлюза позволяет организовать единую точку входа для клиентских приложений, а применение микросервисной архитектуры обеспечивает независимость компонентов, упрощает сопровождение системы и создаёт возможности для её дальнейшего масштабирования.

Спроектированная архитектура позволяет подключать новые сервисы без внесения изменений в ядро системы, что значительно повышает гибкость платформы и облегчает её развитие. Реализованные механизмы аутентификации и авторизации обеспечивают необходимый уровень безопасности и позволяют централизованно контролировать доступ пользователей к ресурсам цифровой экосистемы.

Экономический расчёт показал, что разработка собственного ядра цифровой экосистемы является целесообразной, поскольку позволяет снизить зависимость от сторонних решений, уменьшить затраты на сопровождение распределённой инфраструктуры и обеспечить возможность адаптации системы под конкретные требования организации. Полученные результаты подтверждают практическую эффективность разработанного программного обеспечения и возможность его внедрения в корпоративную информационную среду.

Практическая значимость работы заключается в создании универсальной платформы, которая может служить основой для построения различных цифровых сервисов и информационных систем. Разработанное ядро может использоваться как базовый компонент корпоративных порталов, образовательных платформ, систем электронного документооборота, маркетплейсов и других распределённых программных комплексов.

Список используемых источников

Основные источники:

- 1) Васильев, А.Н. Python на примерах. Практический курс программирования / А.Н. Васильев. – СПб.: Наука и Техника, 2023. – 432 с.
- 2) Клебманн, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клебманн. – СПб.: Питер, 2023. – 672 с.
- 3) Ларман, К. Применение UML и шаблонов проектирования / К. Ларман. – М.: Вильямс, 2020. – 736 с.
- 4) Мартин, Р. Чистый код. Создание, анализ и рефакторинг / Р. Мартин. – СПб.: Питер, 2021. – 464 с.
- 5) Ньюман, С. Создание микросервисов / С. Ньюман. – СПб.: Питер, 2022. – 624 с.
- 6) Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга / К. Ричардсон. – СПб.: Питер, 2021. – 544 с.
- 7) Саммервилл, И. Инженерия программного обеспечения / И. Саммервилл. – М.: Вильямс, 2021. – 928 с.
- 8) Сандерс, К. Docker для профессионалов / К. Сандерс. – СПб.: Питер, 2022. – 416 с.
- 9) Силбершатц, А. Концепции систем баз данных / А. Силбершатц, Г. Корт, С. Сударшан. – М.: Вильямс, 2021. – 1376 с.
- 10) Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. – М.: Вильямс, 2020. – 544 с.

Интернет-ресурсы:

- 11) Документация Docker [Электронный ресурс]. – Docker, 2026. – Режим доступа: <https://docs.docker.com/> (Дата обращения: 20.04.2026).
- 12) Документация FastAPI [Электронный ресурс]. – FastAPI, 2026. – Режим доступа: <https://fastapi.tiangolo.com/> (Дата обращения: 16.04.2026).

					ДП(Р).09.02.07 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		57

13) Документация PostgreSQL [Электронный ресурс]. – PostgresPro, 2026. – Режим доступа: <https://postgrespro.ru/docs/postgresql/current> (Дата обращения: 13.04.2026).

14) Документация Python [Электронный ресурс]. – Python, 2026. – Режим доступа: <https://www.python.org/doc/> (Дата обращения: 02.05.2026).

15) Документация Redis [Электронный ресурс]. – Redis, 2026. – Режим доступа: <https://redis.io/docs/latest/> (Дата обращения: 22.04.2026).

16) Статья «OpenShift как корпоративная версия Kubernetes» [Электронный ресурс]. – Habr, 2020. – Режим доступа: <https://habr.com/ru/companies/redhatrussia/articles/494254/> (Дата обращения: 10.04.2026).

17) Статья «REST API: что это такое и как работает» [Электронный ресурс]. – Яндекс Практикум, 2025. – Режим доступа: <https://practicum.yandex.ru/blog/chto-takoe-rest-api-i-kak-rabotaet/> (Дата обращения: 12.04.2026).

18) Статья «Подробно про JWT» [Электронный ресурс]. – Habr, 2024. – Режим доступа: <https://habr.com/ru/articles/842056/> (Дата обращения: 25.04.2026).

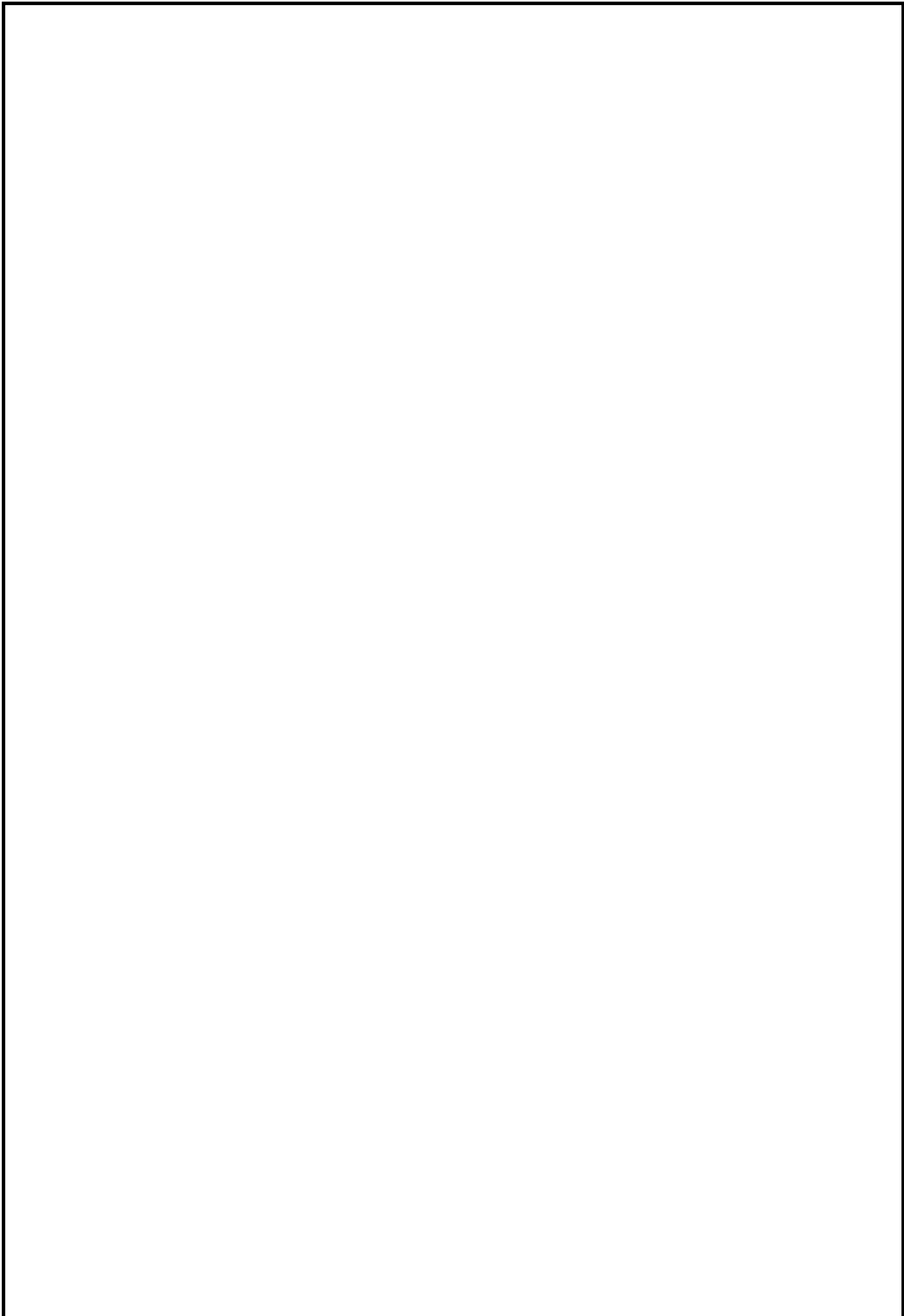
19) Статья «Что такое Amazon Web Services (AWS)» [Электронный ресурс]. – Timeweb, 2021. – Режим доступа: <https://timeweb.cloud/blog/aws> (Дата обращения: 07.04.2026).

20) Статья «Что такое API-шлюзы и Gateway API и почему за ними будущее нативных облачных приложений» [Электронный ресурс]. – Habr, 2023. – Режим доступа: <https://habr.com/ru/companies/flant/articles/770036/> (Дата обращения: 04.04.2026).

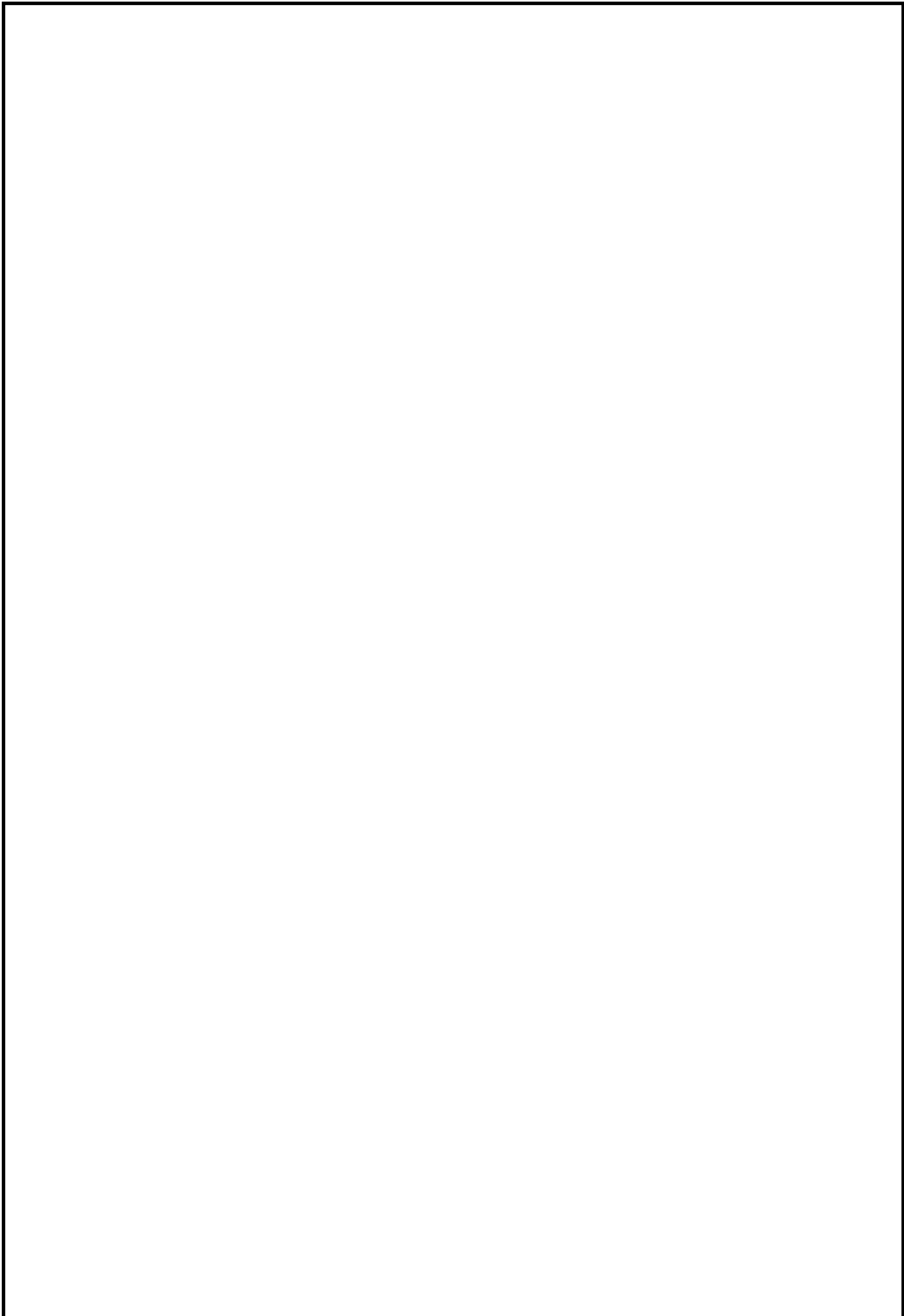
Приложение А
(обязательное)

Графическая часть

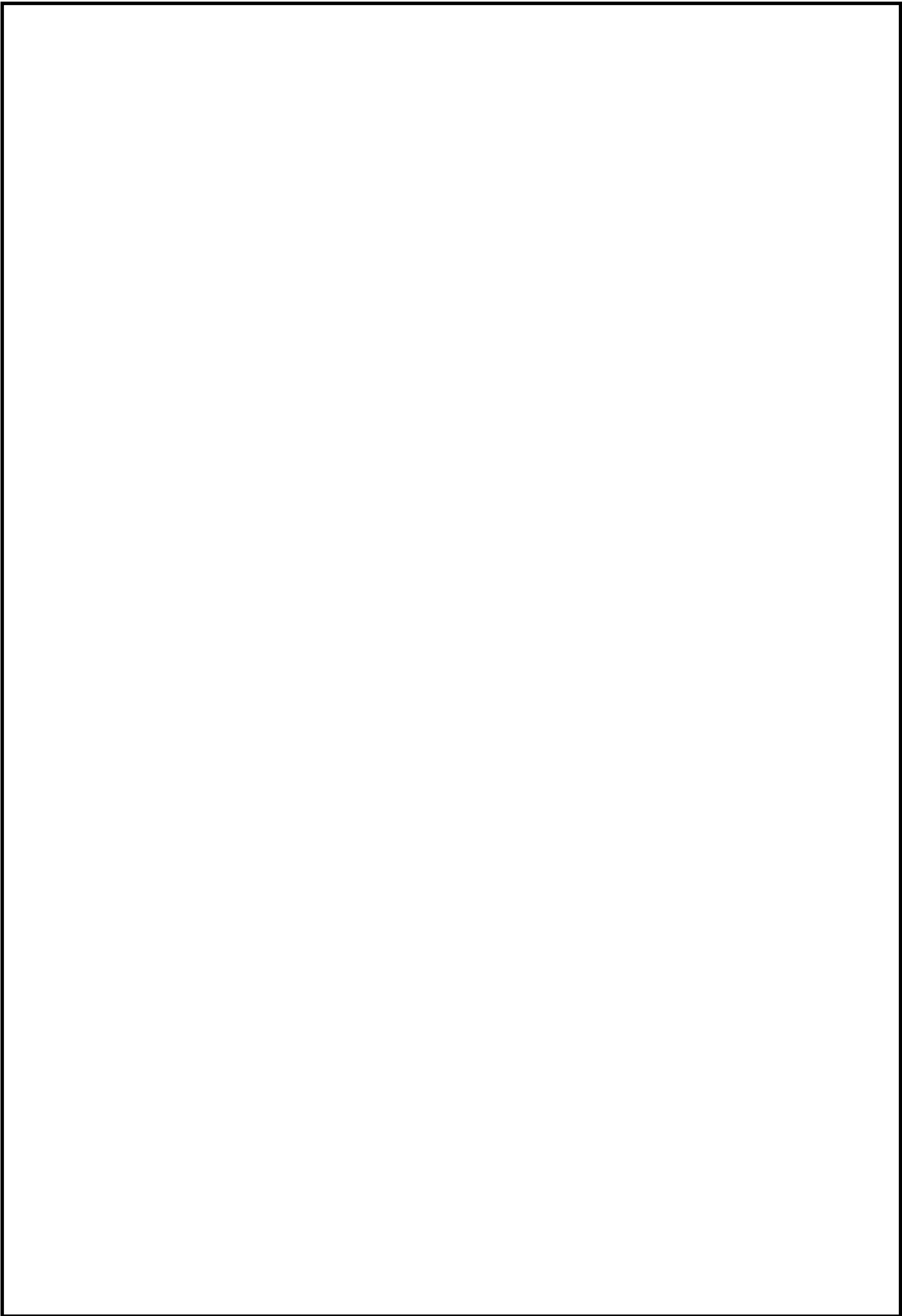
					<i>ДП(Р).09.02.07 ПЗ</i>	<i>Лист</i>
						59
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		



					<i>ДП(Р).09.02.07 ПЗ</i>	<i>Лист</i>
						60
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		



					<i>ДП(Р).09.02.07 ПЗ</i>	<i>Лист</i>
						61
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		



					<i>ДП(Р).09.02.07 ПЗ</i>	<i>Лист</i>
						62
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		

Приложение Б
(обязательное)

Интерфейс программного обеспечения

					<i>ДП(Р).09.02.07 ПЗ</i>	<i>Лист</i>
						63
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		

USER ID
not set

ROLE ID
not set

PERMISSION ID
not set

ACCESS TOKEN
not set

Gateway

PublicPrivateAdmin

Регистрация

Username
demo_user123

Email
demo_user123@example.com

Password
.....

Зарегистрироваться

Вход

Username
demo_user123

Password
.....

Войти

Роль

Role name
ADMIN

Создать рольНазначить роль

Permission

Permission name
Read demo admin endpoint

Создать permissionНазначить permission

Нажмите любую кнопку, чтобы увидеть запрос.

Ответ сервиса появится здесь.

Рисунок 13 – Интерфейс программного обеспечения

Приложение В
(обязательное)

Фрагмент листинга программы

					<i>ДП(Р).09.02.07 ПЗ</i>	<i>Лист</i>
						65
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>		

```
from typing import Any
```

```
import httpx
```

```
from fastapi import Request
```

```
from api_gateway.app.core.config import settings
```

```
from api_gateway.app.services.http_clients import ServiceClient
```

```
from shared.exceptions.api import ForbiddenException, NotFoundException,  
UnauthorizedException
```

```
class GatewayService:
```

```
    """Gateway orchestration between auth, authorization and registry."""
```

```
    def __init__(self) -> None:
```

```
        self.auth_client = ServiceClient(settings.auth_service_url)
```

```
        self.authorization_client = ServiceClient(settings.authorization_service_url)
```

```
        self.registry_client = ServiceClient(settings.service_registry_url)
```

```
    async def authenticate_request(self, request: Request, route_data: dict[str, Any])  
    -> dict[str, Any]:
```

```
        if not route_data["is_protected"]:
```

```
            return {"user_id": None, "roles": [], "permissions": []}
```

```
        auth_header = request.headers.get("Authorization")
```

```
        if not auth_header or not auth_header.startswith("Bearer "):
```

```
            raise UnauthorizedException("Missing bearer token")
```

```
        token = auth_header.removeprefix("Bearer ").strip()
```

```
        validation_response = await self.auth_client.post("/auth/validate", {"token":  
token})
```

					ДП(Р).09.02.07 ПЗ	Лист
						66
Изм.	Лист	№ докум.	Подпись	Дата		

```

return validation_response["data"]

async def authorize_request(self, user_id: str | None, route_data: dict[str, Any]) -
> dict[str, Any]:
    if not route_data["is_protected"]:
        return {"allowed": True, "roles": [], "permissions": []}
    permission = route_data.get("required_permission")
    if not permission:
        return {"allowed": True, "roles": [], "permissions": []}
    resource, action = permission.split(":", maxsplit=1)
    result = await self.authorization_client.post(
        "/authorize/check", {"user_id": user_id, "resource": resource, "action":
action}
    )
    if not result["data"]["allowed"]:
        raise ForbiddenException("Access denied for requested resource")
    return result["data"]

async def resolve_route(self, path: str, method: str) -> dict[str, Any]:
    response = await self.registry_client.get("/services/resolve/route",
params={"path": path, "method": method})
    data = response["data"]
    if data["status"] != "ACTIVE":
        raise NotFoundException("Target service is not available")
    return data

async def forward_request(
    self,
    target_base_url: str,

```

```

path: str,
request: Request,
user_context: dict[str, Any],
authz_context: dict[str, Any],
) -> httpx.Response:
    headers = {
        key: value
        for key, value in request.headers.items()
        if key.lower() not in {"host", "content-length"}
    }
    if user_context.get("user_id"):
        headers["x-user-id"] = user_context["user_id"]
    if authz_context.get("roles"):
        headers["x-user-roles"] = ",".join(authz_context["roles"])
    body = await request.body()
    async with httpx.AsyncClient(timeout=20.0) as client:
        return await client.request(
            request.method,
            f'{target_base_url}{path}',
            headers=headers,
            params=request.query_params,
            content=body,
        )

```